

óé



éó

ñ

```
import logging
```

```

import numpy as np
from collections import Counter

logging.basicConfig(
    level=logging.INFO,
    format="% (asctime)s [% (levelname)s] % (name)s: % (message)s"
)
logger = logging.getLogger(__name__)

```

Regresión lineal y logarítmica

```

class RegresionLineal:
    """
    Regresión Lineal por Descenso de Gradiente.
    Minimiza MSE:  $J = (1/2m) \sum (\hat{y} - y)^2$ 

    Args:
        alpha (float): Tasa de aprendizaje.
        iteraciones (int): Épocas de entrenamiento.
    """

    def __init__(self, alpha=0.01, iteraciones=1000):
        self.alpha = alpha
        self.iteraciones = iteraciones
        self.w = None
        self.b = None

    def fit(self, X, y):
        """
        Ajusta w y b minimizando MSE.

        Args:
            X (np.ndarray): Shape (m, n).
            y (np.ndarray): Shape (m,).

        Returns:
            self
        """
        try:
            if X.shape[0] != y.shape[0]:
                raise ValueError("X e y tienen dimensiones incompatibles.")
            n_samples, n_features = X.shape
            self.w = np.zeros(n_features)
            self.b = 0.0

            for _ in range(self.iteraciones):
                y_pred = np.dot(X, self.w) + self.b          #  $\hat{y} = Xw + b$ 
                dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))

```

```

        db = (1 / n_samples) * np.sum(y_pred - y)
        self.w -= self.alpha * dw
        self.b -= self.alpha * db

        logger.info("RegresionLineal entrenada.")
        return self
    except Exception as e:
        logger.error("Error en RegresionLineal.fit: %s", e)
        raise

def predict(self, X):
    """
    Retorna  $\hat{y} = Xw + b$ .

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Predicciones continuas.
    """
    try:
        if self.w is None:
            raise RuntimeError("Modelo no entrenado.")
        return np.dot(X, self.w) + self.b
    except Exception as e:
        logger.error("Error en RegresionLineal.predict: %s", e)
        raise

```

```

class RegresionLogistica:
    """
    Regresión Logística para clasificación binaria.
    Aplica sigmoide:  $\sigma(z) = 1 / (1 + e^{(-z)})$ 

    Args:
        alpha (float): Tasa de aprendizaje.
        iteraciones (int): Épocas de entrenamiento.
    """

    def __init__(self, alpha=0.01, iteraciones=1000):
        self.alpha = alpha
        self.iteraciones = iteraciones
        self.w = None
        self.b = None

    def _sigmoid(self, z):
        """ $\sigma(z) = 1 / (1 + e^{(-z)}) \rightarrow$  probabilidad en (0, 1)."""
        return 1.0 / (1.0 + np.exp(-z))

    def fit(self, X, y):
        """
        Entrena con descenso de gradiente sobre log-loss.

```

```

Args:
    X (np.ndarray): Shape (m, n).
    y (np.ndarray): Etiquetas binarias {0, 1}, shape (m,).

Returns:
    self
"""
try:
    if X.shape[0] != y.shape[0]:
        raise ValueError("X e y tienen dimensiones incompatibles.")
    n_samples, n_features = X.shape
    self.w = np.zeros(n_features)
    self.b = 0.0

    for _ in range(self.iteraciones):
        z = np.dot(X, self.w) + self.b          # combinación lineal
        y_pred = self._sigmoid(z)               # probabilidades
        dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))
        db = (1 / n_samples) * np.sum(y_pred - y)
        self.w -= self.alpha * dw
        self.b -= self.alpha * db

    logger.info("RegresionLogistica entrenada.")
    return self
except Exception as e:
    logger.error("Error en RegresionLogistica.fit: %s", e)
    raise

def predict(self, X):
    """
    Clasifica usando umbral 0.5:  $\sigma(z) > 0.5 \rightarrow$  clase 1.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        list[int]: Etiquetas {0, 1}.
    """
    try:
        if self.w is None:
            raise RuntimeError("Modelo no entrenado.")
        z = np.dot(X, self.w) + self.b
        return [1 if p > 0.5 else 0 for p in self._sigmoid(z)]
    except Exception as e:
        logger.error("Error en RegresionLogistica.predict: %s", e)
        raise

```

Aplicandolo:

```

import numpy as np
import matplotlib.pyplot as plt

```

```

# --- 1. PREPARACIÓN DE DATOS (Simulando una clase real) ---
# Horas de estudio de 10 alumnos
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9], [10]])
# 0 = Reprobado, 1 = Aprobado
y = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])

# --- 2. EL MODELO (Usando la lógica que desarrollamos antes) ---
modelo = RegresionLogistica(alpha=0.1, iteraciones=500)
modelo.fit(X, y)

# --- 3. PREDICCIÓN APLICADA ---
# ¿Qué pasa con un alumno que estudia 5.5 horas?
horas_nuevas = np.array([[5.5]])
prediccion = modelo.predict(horas_nuevas)

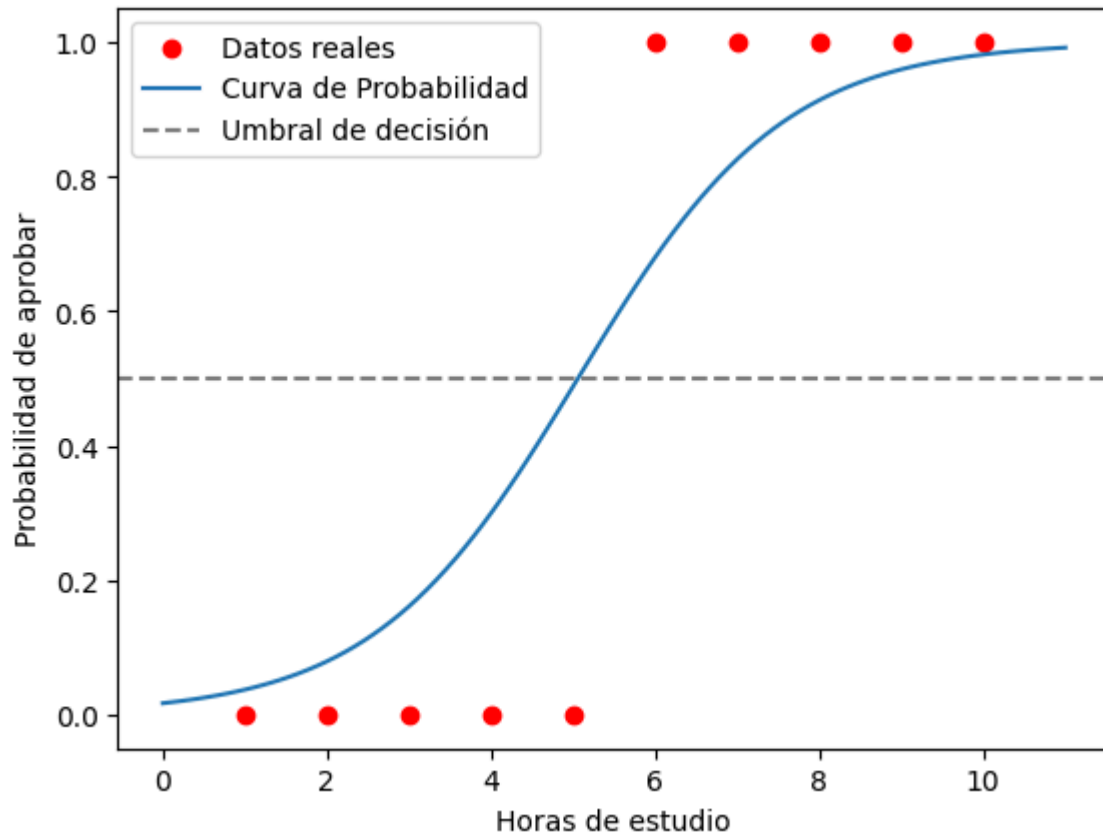
resultado = "Aprobado" if prediccion[0] == 1 else "Reprobado"
print(f"Resultado para 5.5 horas: {resultado}")

# --- 4. VISUALIZACIÓN (Crucial en experimentación) ---
plt.scatter(X, y, color='red', label='Datos reales')
# Generamos puntos para ver la curva sigmoide
X_curva = np.linspace(0, 11, 100).reshape(-1, 1)
z = np.dot(X_curva, modelo.w) + modelo.b
probabilidades = 1 / (1 + np.exp(-z))

plt.plot(X_curva, probabilidades, label='Curva de Probabilidad')
plt.axhline(y=0.5, color='gray', linestyle='--', label='Umbral de decisión')
plt.xlabel("Horas de estudio")
plt.ylabel("Probabilidad de aprobar")
plt.legend()
plt.show()

```

Resultado para 5.5 horas: Aprobado



SVM

```
import numpy as np
from sklearn.svm import SVC # La versión profesional

class SVM_Manual:
    """
    SVM lineal con pérdida Hinge y regularización L2.
     $J(w) = \lambda ||w||^2 + \max(0, 1 - y(wx - b))$ 

    Args:
        learning_rate (float): Tasa de aprendizaje.
        lambda_param (float): Regularización L2.
        n_iters (int): Iteraciones de entrenamiento.
    """

    def __init__(self, learning_rate=0.001, lambda_param=0.01, n_iters=1000):
        self.lr = learning_rate
        self.lambda_param = lambda_param
        self.n_iters = n_iters
        self.w = None
        self.b = None

    def fit(self, X, y):
        """
```

Entrena minimizando la pérdida Hinge punto a punto.

Args:

X (np.ndarray): Shape (m, n).
y (np.ndarray): Etiquetas {0,1} o {-1,1}, shape (m,).

Returns:

self

"""

try:

if X.shape[0] != y.shape[0]:
 raise ValueError("X e y tienen dimensiones incompatibles.")
y_ = np.where(y <= 0, -1, 1) # convertir a {-1, 1}
n_samples, n_features = X.shape
self.w = np.zeros(n_features)
self.b = 0.0

for _ in range(self.n_iters):
 for idx, x_i in enumerate(X):
 # Condición de margen: $y_i(w \cdot x_i - b) \geq 1$
 margin = y_[idx] * (np.dot(x_i, self.w) - self.b)
 if margin >= 1:
 self.w -= self.lr * (2 * self.lambda_param * self.w)
 else:
 self.w -= self.lr * (
 2 * self.lambda_param * self.w
 - np.dot(x_i, y_[idx])
)
 self.b -= self.lr * y_[idx]

logger.info("SVM_Manual entrenada.")

return self

except Exception as e:

logger.error("Error en SVM_Manual.fit: %s", e)

raise

def predict(self, X):

"""

Retorna sign($w \cdot x - b$).

Args:

X (np.ndarray): Shape (m, n).

Returns:

np.ndarray: Etiquetas {-1, 1}.

"""

try:

if self.w is None:
 raise RuntimeError("Modelo no entrenado.")
return np.sign(np.dot(X, self.w) - self.b)

except Exception as e:

logger.error("Error en SVM_Manual.predict: %s", e)

raise

Para probar la implementación manual de SVM, se necesita generar datos sintéticos que sepamos que son "separables" para verificar si el algoritmo encuentra los pesos correctos.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# 1. Generamos datos de prueba (2 grupos de puntos)
# Usamos etiquetas -1 y 1 porque así lo definimos en el algoritmo manual
X, y = make_blobs(n_samples=50, n_features=2, centers=2, cluster_std=1.05,
random_state=40)
y = np.where(y == 0, -1, 1)

# 2. Instanciamos y entrenamos el modelo manual
modelo_svm = SVM_Manual(learning_rate=0.001, lambda_param=0.01, n_iters=1000)
modelo_svm.fit(X, y)

print(f"Pesos finales (w): {modelo_svm.w}")
print(f"Sesgo final (b): {modelo_svm.b}")

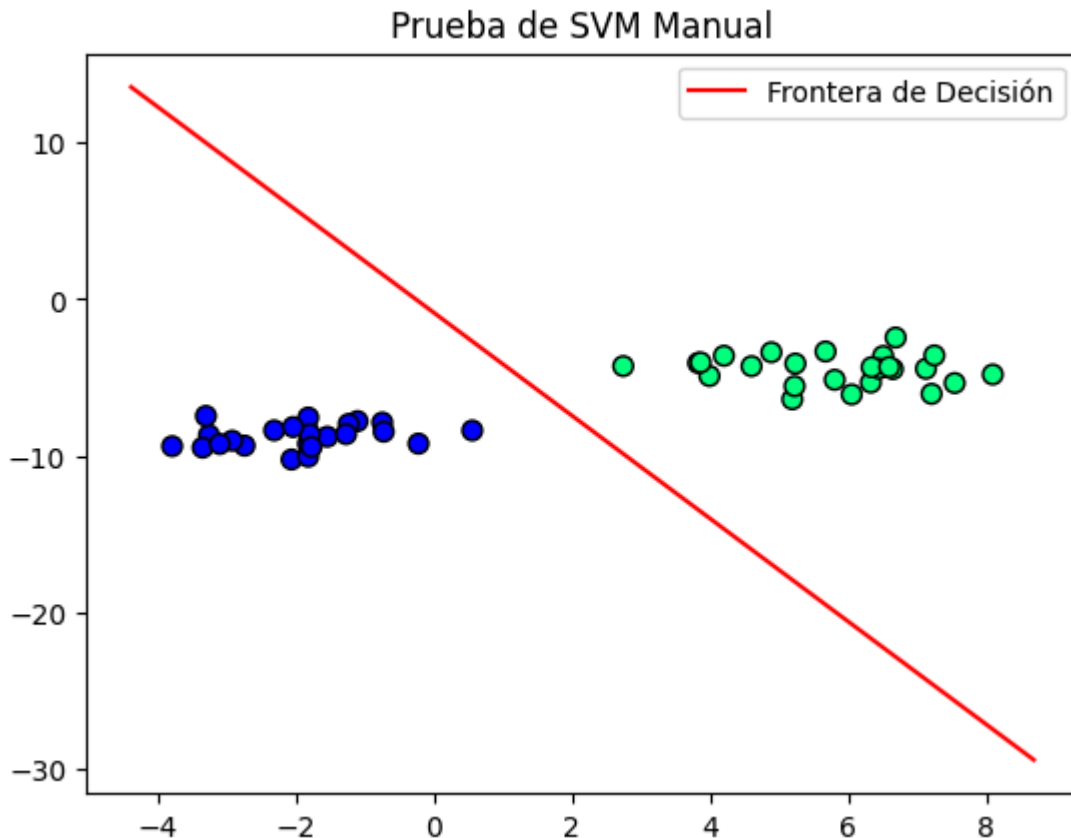
# 3. Función para visualizar el resultado
def visualizar_svm(X, y, model):
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='winter', s=50, edgecolors='k')
    ax = plt.gca()
    xlim = ax.get_xlim()

    # Calculamos la línea de decisión:  $w_1x_1 + w_2x_2 - b = 0$ 
    # Despejando  $x_2$ :  $x_2 = (-w_1x_1 + b) / w_2$ 
    x1 = np.linspace(xlim[0], xlim[1], 100)
    x2 = (model.b - model.w[0] * x1) / model.w[1]

    plt.plot(x1, x2, 'r-', label="Frontera de Decisión")
    plt.title("Prueba de SVM Manual")
    plt.legend()
    plt.show()

visualizar_svm(X, y, modelo_svm)

Pesos finales (w): [0.58977016 0.17946483]
Sesgo final (b): -0.15200000000000001
```

Análisis de los Resultados

1. Separación Lineal: La línea roja (tu frontera de decisión) ha logrado separar perfectamente el grupo azul del grupo verde. Esto indica que el hiperplano encontrado es válido.
2. Los Pesos (w): Tus pesos finales no son cero, lo que significa que el algoritmo actualizó los parámetros mediante el gradiente de forma efectiva.
3. El Sesgo (b): El valor de b es el que desplaza la línea fuera del origen para que pueda quedar justo en medio de las dos "nubes" de puntos.
4. Optimización del Margen: Nota que la línea no está "pegada" a un grupo, sino que intenta mantener una distancia prudente de ambos. Eso es exactamente lo que busca una SVM: maximizar el margen.

k-Nearest Neighbors (k-NN)

```
import numpy as np
from collections import Counter

class KNN:
    """
    K-Nearest Neighbors por distancia Euclidiana y votación mayoritaria.
    Complejidad:  $O(1)$  entrenamiento,  $O(m \cdot n)$  predicción.

    Args:
        k (int): Número de vecinos.
```

```

"""

def __init__(self, k=3):
    self.k = k
    self.X_train = None
    self.y_train = None

def fit(self, X, y):
    """
    Memoriza los datos (lazy learning).

    Args:
        X (np.ndarray): Shape (m, n).
        y (np.ndarray): Shape (m,).

    Returns:
        self
    """
    try:
        self.X_train = X
        self.y_train = y
        logger.info("KNN listo con %d muestras, k=%d.", len(y), self.k)
        return self
    except Exception as e:
        logger.error("Error en KNN.fit: %s", e)
        raise

def predict(self, X):
    """
    Predice la clase de cada muestra en X.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Etiquetas predichas.
    """
    try:
        if self.X_train is None:
            raise RuntimeError("Modelo no entrenado.")
        return np.array([self._predict(x) for x in X])
    except Exception as e:
        logger.error("Error en KNN.predict: %s", e)
        raise

def _predict(self, x):
    """Clasifica x por votación entre los k vecinos más cercanos."""
    #  $d(x, x_i) = \sqrt{\sum (x - x_i)^2}$ 
    distances = [
        np.sqrt(np.sum((x - x_train) ** 2))
        for x_train in self.X_train
    ]
    k_indices = np.argsort(distances)[: self.k]

```

```
k_labels = [self.y_train[i] for i in k_indices]
return Counter(k_labels).most_common(1)[0][0]
```

Puntos clave:

- Vectorización con NumPy: En la línea `np.sqrt(np.sum((x - x_train)**2))`, estamos aplicando el formalismo matemático de la distancia euclidiana. Se prefieren operaciones vectorizadas sobre ciclos for anidados (eficiencia de memoria y caché).
- argsort vs sort: No nos interesa la distancia en sí para el resultado final, sino el índice del dato original que generó esa distancia, para poder buscar su etiqueta en `y_train`.
- El rol de Counter: Es la forma más limpia en Python de implementar la "Votación de Pluralidad" que pusimos en el pseudocódigo.
- Escalabilidad: Por cada punto nuevo en `X_new`, tenemos que recorrer todo `X_train`. Si el dataset fuera de 1 millón de registros, el tiempo de respuesta (latencia) sería inaceptable para una aplicación real.

Desafío

Modificar el código para que soporte distancia de Manhattan (

) o que implementen un esquema de pesos, donde los vecinos más cercanos tengan más influencia en el voto que los que están más lejos.

```
if __name__ == "__main__":
    # Datos sintéticos: [característica 1, característica 2]
    X_train = np.array([[1, 2], [2, 3], [1.5, 1.8], [7, 8], [8, 9], [7,
6.5]])
    y_train = np.array([0, 0, 0, 1, 1, 1]) # Clase 0 y Clase 1

    clf = KNN(k=3)
    clf.fit(X_train, y_train)

    # Un nuevo punto que queremos clasificar
    X_new = np.array([[4, 5]])
    predicciones = clf.predict(X_new)

    print(f"Predicciones para los nuevos puntos: Clase {predicciones[0]}")
```

Predicciones para los nuevos puntos: Clase 0

```
# --- CÓDIGO DE VISUALIZACIÓN RÁPIDA ---
import matplotlib.pyplot as plt

# 1. Datos pequeños (mismo formato de tu fit)
X_train = np.array([[1, 2], [2, 3], [1.5, 1.8], [7, 8], [8, 9], [7, 6.5]])
y_train = np.array([0, 0, 0, 1, 1, 1])
```

```

# 2. Instanciar y "entrenar" con tu clase
clf = KNN(k=3)
clf.fit(X_train, y_train)

# 3. Punto nuevo a clasificar
X_nuevo = np.array([[4, 5]])
prediccion = clf.predict(X_nuevo)

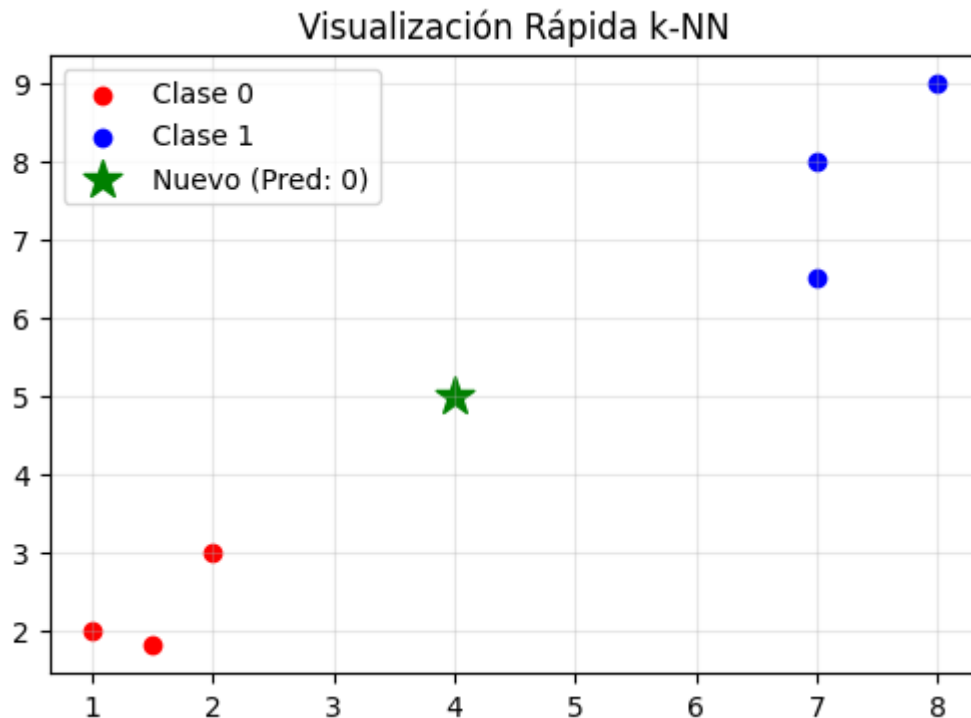
# 4. Dibujo directo
plt.figure(figsize=(6, 4))

# Dibujamos los puntos de entrenamiento (Clase 0 rojos, Clase 1 azules)
plt.scatter(X_train[y_train==0, 0], X_train[y_train==0, 1], c='red',
            label='Clase 0')
plt.scatter(X_train[y_train==1, 0], X_train[y_train==1, 1], c='blue',
            label='Clase 1')

# Dibujamos el punto nuevo como una estrella verde
plt.scatter(X_nuevo[:, 0], X_nuevo[:, 1], c='green', marker='*', s=200,
            label=f'Nuevo (Pred: {prediccion[0]})')

plt.title("Visualización Rápida k-NN")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```



Arbol de Decisión

```

import numpy as np
from collections import Counter

class Nodo:
    """Nodo de árbol de decisión (pregunta o hoja)."""

    def __init__(
        self, feature=None, threshold=None,
        left=None, right=None, value=None
    ):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value


class ArbolDecision:
    """
    Árbol de Decisión CART usando Entropía de Shannon.
     $IG = H(\text{padre}) - \text{promedio\_ponderado}(H(\text{hijos}))$ 
     $H(S) = -\sum p_i \cdot \log_2(p_i)$ 

    Args:
        max_depth (int): Profundidad máxima.
    """

    def __init__(self, max_depth=5):
        self.max_depth = max_depth
        self.root = None

    def fit(self, X, y):
        """
        Construye el árbol recursivamente.

        Args:
            X (np.ndarray): Shape (m, n).
            y (np.ndarray): Etiquetas enteras, shape (m,).

        Returns:
            self
        """
        try:
            if X.shape[0] != y.shape[0]:
                raise ValueError("X e y tienen dimensiones incompatibles.")
            self.root = self._crear_arbol(X, y, depth=0)
            logger.info(
                "ArbolDecision construido, max_depth=%d.", self.max_depth
            )
            return self
        except Exception as e:
            logger.error("Error en ArbolDecision.fit: %s", e)
            raise

```

```

def _crear_arbol(self, X, y, depth):
    """Construcción recursiva; retorna Nodo hoja o de decisión."""
    n_samples, n_features = X.shape
    if depth >= self.max_depth or len(np.unique(y)) == 1:
        return Nodo(value=Counter(y).most_common(1)[0][0])

    best_feat, best_thresh = self._mejor_criterio(X, y, n_features)
    left_idx = X[:, best_feat] <= best_thresh
    izq = self._crear_arbol(X[left_idx], y[left_idx], depth + 1)
    der = self._crear_arbol(X[~left_idx], y[~left_idx], depth + 1)
    return Nodo(
        feature=best_feat, threshold=best_thresh,
        left=izq, right=der
    )

def _mejor_criterio(self, X, y, n_features):
    """Busca la característica y umbral con mayor ganancia de info."""
    best_gain, split_idx, split_thresh = -1, None, None
    for feat_idx in range(n_features):
        for threshold in np.unique(X[:, feat_idx]):
            gain = self._informacion_ganada(
                X[:, feat_idx], y, threshold
            )
            if gain > best_gain:
                best_gain = gain
                split_idx = feat_idx
                split_thresh = threshold
    return split_idx, split_thresh

def _informacion_ganada(self, col, y, threshold):
    """IG = H(padre) - (nL/n)H(izq) - (nR/n)H(der)."""
    left_idx = col <= threshold
    right_idx = ~left_idx
    if len(y[left_idx]) == 0 or len(y[right_idx]) == 0:
        return 0
    n = len(y)
    n_l, n_r = len(y[left_idx]), len(y[right_idx])
    return (
        self._entropia(y)
        - (n_l / n) * self._entropia(y[left_idx])
        - (n_r / n) * self._entropia(y[right_idx])
    )

def _entropia(self, y):
    """ $H(S) = -\sum p_i \cdot \log_2(p_i)$ ."""
    props = np.bincount(y) / len(y)
    return -np.sum([p * np.log2(p) for p in props if p > 0])

def predict(self, X):
    """
    Recorre el árbol para clasificar cada muestra.
    """

```

```

Args:
    X (np.ndarray): Shape (m, n).

Returns:
    np.ndarray: Etiquetas predichas.
"""
try:
    if self.root is None:
        raise RuntimeError("Modelo no entrenado.")
    return np.array([self._recorrer(x, self.root) for x in X])
except Exception as e:
    logger.error("Error en ArbolDecision.predict: %s", e)
    raise

def _recorrer(self, x, nodo):
    """Recorre recursivamente hasta una hoja."""
    if nodo.value is not None:
        return nodo.value
    if x[nodo.feature] <= nodo.threshold:
        return self._recorrer(x, nodo.left)
    return self._recorrer(x, nodo.right)

```

Puntos clave:

- La clase Nodo: Es fundamental que se entienda que el árbol no es un arreglo, sino una estructura de datos enlazada. Cada nodo o es una "pregunta" (feature y threshold) o es una "respuesta" (value).
- Recursividad: La función de `_crear_arbol` se llama a sí mismo para construir las ramas. Es una aplicación real y poderosa de lo que ven en sus clases de diseño y análisis de algoritmos.

-La Entropía: El método `_entropia` usa `np.bincount` para contar cuántos elementos hay de cada clase, tal cual la fórmula

.

- Greedy Search (Búsqueda Ávida): Es importante comprender que el algoritmo prueba todos los umbrales posibles para todas las características en cada paso para encontrar el mejor. Esto explica por qué el entrenamiento es .

```

# --- Ejecución y Dibujo Rápido ---
import matplotlib.pyplot as plt

if __name__ == "__main__":
    # Dataset simple: [Estatura, Peso] -> 0: Gato, 1: Perro
    X_train = np.array([[15, 4], [18, 5], [20, 6], [35, 20], [40, 25], [45,
30]])
    y_train = np.array([0, 0, 0, 1, 1, 1])

    clf = ArbolDecision(max_depth=2)
    clf.fit(X_train, y_train)

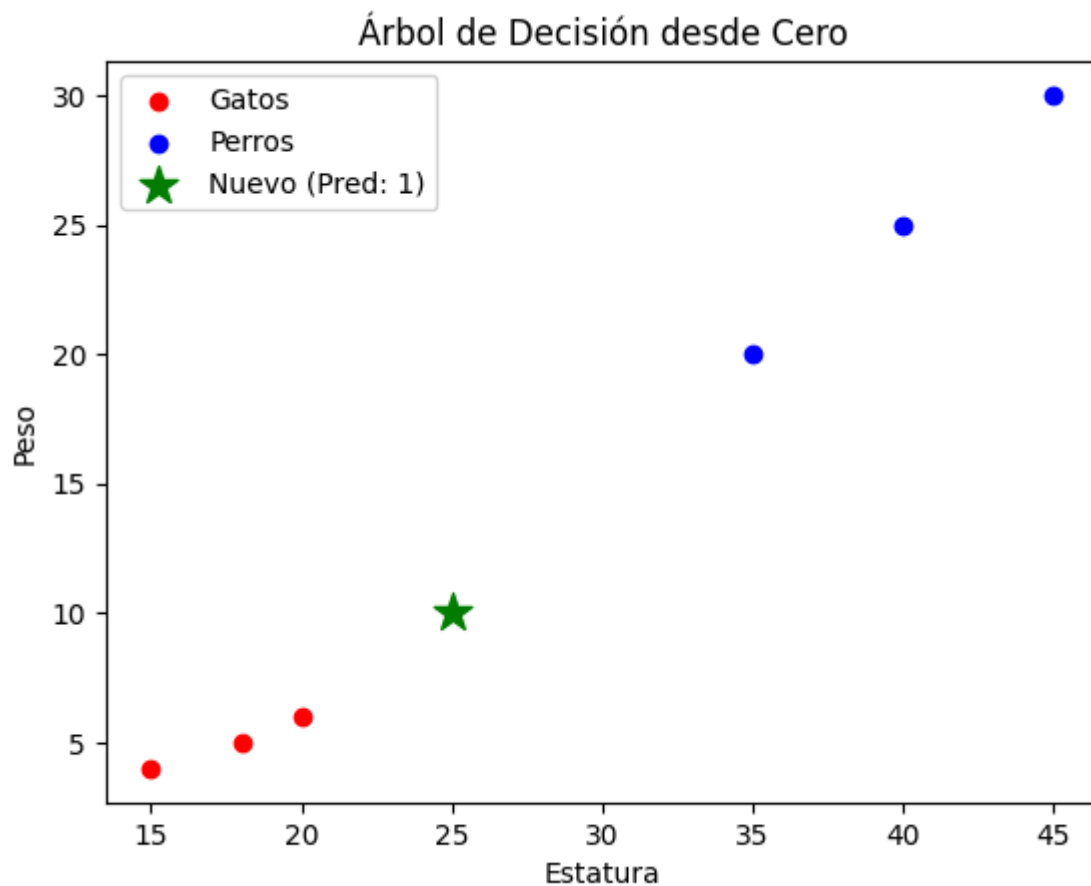
```

```

# Punto nuevo
X_test = np.array([[25, 10]])
pred = clf.predict(X_test)

# Gráfica
plt.scatter(X_train[y_train==0, 0], X_train[y_train==0, 1], c='red',
label='Gatos')
plt.scatter(X_train[y_train==1, 0], X_train[y_train==1, 1], c='blue',
label='Perros')
plt.scatter(X_test[:, 0], X_test[:, 1], c='green', marker='*', s=200,
label=f'Nuevo (Pred: {pred[0]})')
plt.xlabel("Estatura")
plt.ylabel("Peso")
plt.legend()
plt.title("Árbol de Decisión desde Cero")
plt.show()

```



Con nodos

```

import numpy as np
from collections import Counter
import matplotlib.pyplot as plt
import networkx as nx

```



```

import pydot
from networkx.drawing.nx_pydot import graphviz_layout

# =====
# 1. Definición de Clases con Estado de Datos
# =====

class Nodo:
    def __init__(self, feature=None, threshold=None, left=None, right=None,
value=None, X_indices=None, class_counts=None):
        self.feature = feature      # Índice de la característica de corte
        self.threshold = threshold # Valor de corte
        self.left = left           # Sub-árbol izquierdo
        self.right = right         # Sub-árbol derecho
        self.value = value         # Clase final (si es hoja)
        self.X_indices = X_indices # Índices de X_train en este nodo
        self.class_counts = class_counts # Conteo de clases en este nodo
        (ej: {0: 3, 1: 0})

class ArbolDecisionVisual:
    def __init__(self, max_depth=5, feature_names=None, class_names=None):
        self.max_depth = max_depth
        self.root = None
        self.feature_names = feature_names # Ej: ['Estatura', 'Peso']
        self.class_names = class_names     # Ej: ['Gato', 'Perro']
        self.graph = nx.DiGraph()          # Grafo de NetworkX
        self.node_labels = {}               # Etiquetas de texto para los
nodos
        self.node_colors = {}              # Colores para los nodos

# =====
# 2. Métodos de Entrenamiento (fit) con Generación de Visualización
# =====

    def fit(self, X, y):
        # Guardamos las referencias a los datos completos para la
visualización
        self.X_train = X
        self.y_train = y
        indices_iniciales = np.arange(X.shape[0])
        self.root = self._crear_arbol_visual(X, y, depth=0,
X_indices=indices_iniciales, node_id=0)
        print("Árbol entrenado y visualización generada.")

    def _crear_arbol_visual(self, X, y, depth, X_indices, node_id):
        """Creación recursiva del árbol que también construye el grafo de
visualización"""
        n_samples = len(X_indices)
        if n_samples == 0: return None

        # Contar clases en este nodo
        y_nodo = y[X_indices]
        counts = Counter(y_nodo)

```

```

n_labels = len(counts)

# Determinar si es hoja o pregunta
es_hoja = (depth >= self.max_depth or n_labels == 1)

if es_hoja:
    leaf_value = counts.most_common(1)[0][0]
    nodo_actual = Nodo(value=leaf_value, X_indices=X_indices,
class_counts=counts)
    self._añadir_nodo_al_grafo(node_id, nodo_actual, es_hoja=True)
    return nodo_actual

# Encontrar el mejor criterio (usando entropía, igual que antes)
best_feat, best_thresh = self._mejor_criterio(X[X_indices], y_nodo,
X.shape[1])

# Particionar los índices originales
left_mask = X[X_indices, best_feat] <= best_thresh
right_mask = ~left_mask
left_indices = X_indices[left_mask]
right_indices = X_indices[right_mask]

# Crear nodo pregunta y añadir al grafo ANTES de la recursión
nodo_actual = Nodo(feature=best_feat, threshold=best_thresh,
X_indices=X_indices, class_counts=counts)
self._añadir_nodo_al_grafo(node_id, nodo_actual, es_hoja=False)

# Recursión para hijos (IDs únicos para NetworkX)
id_izq = 2 * node_id + 1
id_der = 2 * node_id + 2

nodo_actual.left = self._crear_arbol_visual(X, y, depth + 1,
left_indices, id_izq)
nodo_actual.right = self._crear_arbol_visual(X, y, depth + 1,
right_indices, id_der)

# Añadir aristas (Si/No)
if nodo_actual.left: self.graph.add_edge(node_id, id_izq, label="<="
umbral")
if nodo_actual.right: self.graph.add_edge(node_id, id_der, label=">
umbral")

return nodo_actual

def _añadir_nodo_al_grafo(self, node_id, nodo, es_hoja):
    """Genera la etiqueta de texto con datos reales y define el color del
nodo"""

    # 1. Crear la etiqueta de datos (con conteo de clases)
    samples_text = f"Muestras: {len(nodo.X_indices)}\n"
    counts_text = "Clases: "
    for i, class_name in enumerate(self.class_names):
        counts_text += f"{class_name}:{nodo.class_counts.get(i, 0)} "

```

```

data_text = samples_text + counts_text.strip()

# 2. Crear la etiqueta de decisión o clase final
if es_hoja:
    final_class_name = self.class_names[nodo.value]
    label = f"HOJA:\n{final_class_name}\n({data_text})"
    # Color basado en la clase mayoritaria (Gato=Rojo, Perro=Azul)
    color = 'lightcoral' if nodo.value == 0 else 'lightskyblue'
else:
    f_name = self.feature_names[nodo.feature]
    label = f"PREGUNTA:\n{f_name} <= {nodo.threshold}\n({data_text})"
    # Color neutro para preguntas
    color = 'lightgray'

self.node_labels[node_id] = label
self.node_colors[node_id] = color
self.graph.add_node(node_id, label=label)

# =====
# 3. Métodos de Predicción (predict) con Resaltado de Ruta
# =====

def predict_con_ruta(self, x_nuevo):
    """Clasifica un punto y devuelve la ruta de IDs de nodos que tomó"""
    ruta = []
    prediccion = self._recorrer_arbol_con_ruta(x_nuevo, self.root,
node_id=0, ruta=ruta)
    return prediccion, ruta

def _recorrer_arbol_con_ruta(self, x, nodo, node_id, ruta):
    if nodo is None: return None
    ruta.append(node_id) # Registrar este nodo en la ruta

    if nodo.value is not None:
        return nodo.value # Encontramos la hoja, fin

    # Decidir rama (usando IDs únicos para NetworkX)
    if x[nodo.feature] <= nodo.threshold:
        id_izq = 2 * node_id + 1
        return self._recorrer_arbol_con_ruta(x, nodo.left, id_izq, ruta)
    else:
        id_der = 2 * node_id + 2
        return self._recorrer_arbol_con_ruta(x, nodo.right, id_der, ruta)

# =====
# 4. Métodos Auxiliares de Visualización y Lógica
# =====

def dibujar_arbol(self, title="Árbol de Decisión con Datos y Ruta",
ruta_resaltada=None):
    """Dibuja el grafo jerárquico. Opcionalmente resalta una ruta de
predicción."""

```

```

if not self.root:
    print("Error: El árbol no ha sido entrenado.")
    return

try:
    # Layout jerárquico 'dot' (raíz arriba, hojas abajo)
    pos = graphviz_layout(self.graph, prog='dot')
except ImportError:
    print("Error: Se requiere 'pydot' y 'Graphviz' instalados.")
    return

plt.figure(figsize=(12, 10))
plt.title(title, fontsize=16)

# 1. Definir estilos de nodos (resaltar ruta si existe)
colors_list = [self.node_colors[n] for n in self.graph.nodes]
edgecolors_list = ['black'] * len(self.graph.nodes)
linewidths_list = [1] * len(self.graph.nodes)

if ruta_resaltada:
    for node_id in ruta_resaltada:
        # Encontrar el índice del node_id en la lista ordenada de
        # nodos
        idx = list(self.graph.nodes).index(node_id)
        edgecolors_list[idx] = 'green' # Borde verde para la ruta
        linewidths_list[idx] = 4      # Borde grueso

# 2. Dibujar nodos
nx.draw(self.graph, pos, with_labels=False, node_size=5000,
        node_color=colors_list, edgecolors=edgecolors_list,
        linewidths=linewidths_list,
        node_shape='s', arrows=True)

nx.draw_networkx_labels(self.graph, pos, labels=self.node_labels,
font_size=9, font_weight='bold')

# 3. Dibujar aristas con etiquetas (<= umbral / > umbral)
edge_labels = nx.get_edge_attributes(self.graph, 'label')
nx.draw_networkx_edge_labels(self.graph, pos,
edge_labels=edge_labels, font_size=11, font_color='darkgreen')

plt.axis('off')
plt.tight_layout()
plt.show()

# --- Tus métodos de entropía y mejor criterio originales (sin cambios)
---
def _mejor_criterio(self, X, y, n_features):
    best_gain = -1
    split_idx, split_thresh = None, None
    for feat_idx in range(n_features):
        thresholds = np.unique(X[:, feat_idx])
        for threshold in thresholds:

```

```

        gain = self._informacion_ganada(X[:, feat_idx], y, threshold)
        if gain > best_gain:
            best_gain = gain
            split_idx = feat_idx
            split_thresh = threshold
        return split_idx, split_thresh

def _informacion_ganada(self, feature_col, y, threshold):
    parent_entropy = self._entropia(y)
    left_idx = feature_col <= threshold
    right_idx = ~left_idx
    if len(y[left_idx]) == 0 or len(y[right_idx]) == 0: return 0
    n = len(y)
    n_l, n_r = len(y[left_idx]), len(y[right_idx])
    child_entropy = (n_l/n) * self._entropia(y[left_idx]) + (n_r/n) *
self._entropia(y[right_idx])
    return parent_entropy - child_entropy

def _entropia(self, y):
    n = len(y)
    if n == 0: return 0
    counts = Counter(y)
    probs = [count / n for count in counts.values()]
    return -np.sum([p * np.log2(p) for p in probs if p > 0])

# =====
# 5. Bloque Principal de Ejecución
# =====

if __name__ == "__main__":
    # 1. Dataset simple (Gatos vs Perros)
    # [Estatura, Peso]
    X_train = np.array([[15, 4], [18, 5], [20, 6], [35, 20], [40, 25], [45,
30]])
    y_train = np.array([0, 0, 0, 1, 1, 1]) # 0: Gato (Rojo), 1: Perro (Azul)

    # 2. Configurar y Entrenar el Árbol Visual
    # Definimos nombres claros para las características y clases
    clf = ArbolDecisionVisual(max_depth=3,
                              feature_names=['Estatura', 'Peso'],
                              class_names=['Gato', 'Perro'])
    clf.fit(X_train, y_train)

    # 3. Clasificar un punto nuevo y obtener su ruta
    x_nuevo = np.array([25, 10]) # Estatura 25, Peso 10
    pred, ruta_tomada = clf.predict_con_ruta(x_nuevo)
    print(f"\nPredicción para {x_nuevo}: {clf.class_names[pred]}")
    print(f"Ruta de nodos tomada: {ruta_tomada}")

    # 4. ;DIBUJAR EL ÁRBOL CON LA RUTA RESALTADA!
    clf.dibujar_arbol(title=f"Árbol de Decisión: Ruta para
{clf.class_names[pred]} (en verde)",
                      ruta_resaltada=ruta_tomada)

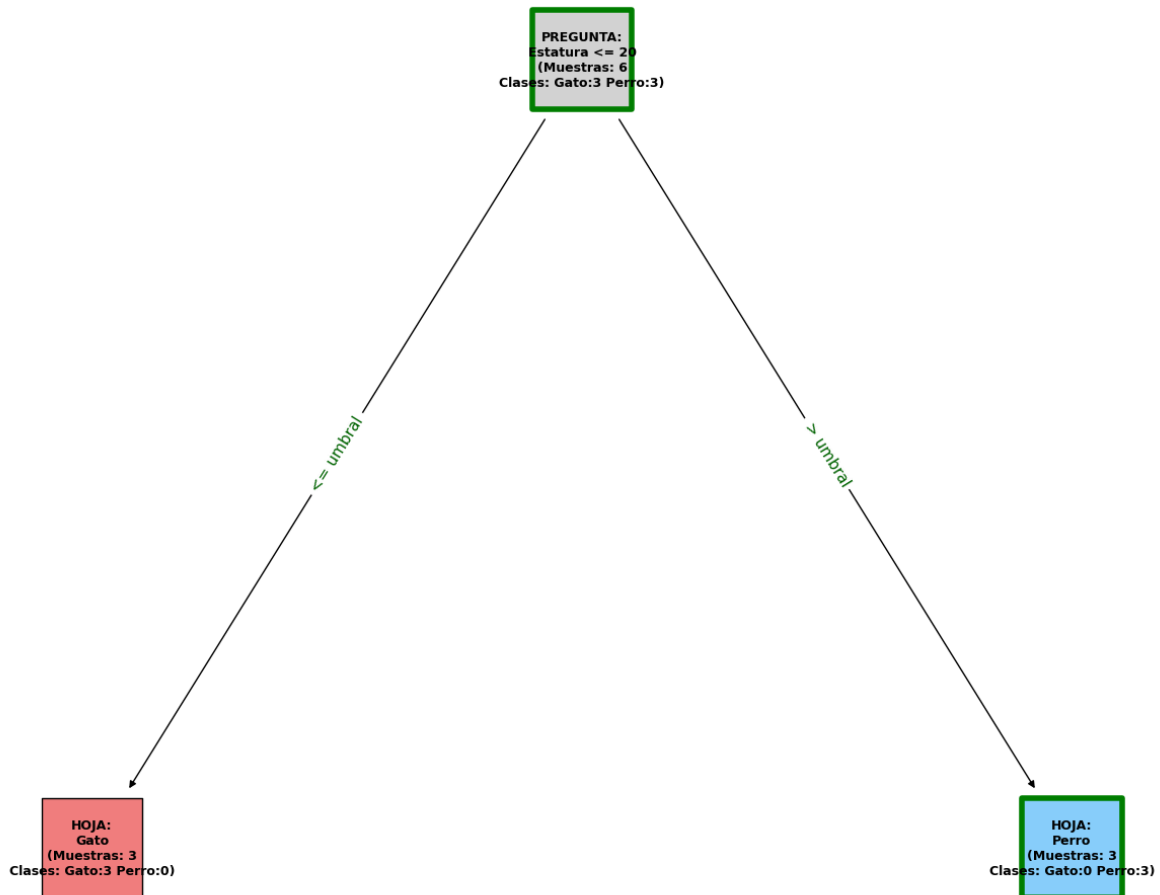
```

Árbol entrenado y visualización generada.

Predicción para [25 10]: Perro

Ruta de nodos tomada: [0, 2]

Árbol de Decisión: Ruta para Perro (en verde)



Otra opción

```
import numpy as np
from collections import Counter

class Nodo:
    def __init__(self, feature=None, threshold=None, left=None, right=None,
value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

class ArbolDecision:
```

```

def __init__(self, max_depth=5):
    self.max_depth = max_depth
    self.root = None

def fit(self, X, y):
    self.root = self._crear_arbol(X, y, depth=0)

def _crear_arbol(self, X, y, depth):
    n_samples, n_features = X.shape
    n_labels = len(np.unique(y))

    if depth >= self.max_depth or n_labels == 1:
        leaf_value = Counter(y).most_common(1)[0][0]
        return Nodo(value=leaf_value)

    best_feat, best_thresh = self._mejor_criterio(X, y, n_features)

    left_idx = X[:, best_feat] <= best_thresh
    right_idx = X[:, best_feat] > best_thresh

    izq = self._crear_arbol(X[left_idx], y[left_idx], depth + 1)
    der = self._crear_arbol(X[right_idx], y[right_idx], depth + 1)
    return Nodo(feature=best_feat, threshold=best_thresh, left=izq,
right=der)

def _mejor_criterio(self, X, y, n_features):
    best_gain = -1
    split_idx, split_thresh = None, None
    for feat_idx in range(n_features):
        thresholds = np.unique(X[:, feat_idx])
        for threshold in thresholds:
            gain = self._informacion_ganada(X[:, feat_idx], y, threshold)
            if gain > best_gain:
                best_gain = gain
                split_idx = feat_idx
                split_thresh = threshold
    return split_idx, split_thresh

def _informacion_ganada(self, feature_col, y, threshold):
    parent_entropy = self._entropia(y)
    left_idx = feature_col <= threshold
    right_idx = feature_col > threshold
    if len(y[left_idx]) == 0 or len(y[right_idx]) == 0: return 0
    n = len(y)
    n_l, n_r = len(y[left_idx]), len(y[right_idx])
    child_entropy = (n_l/n) * self._entropia(y[left_idx]) + (n_r/n) *
self._entropia(y[right_idx])
    return parent_entropy - child_entropy

def _entropia(self, y):
    proportions = np.bincount(y) / len(y)
    return -np.sum([p * np.log2(p) for p in proportions if p > 0])

```

```

def predict(self, X):
    return np.array([self._recorrer_arbol(x, self.root) for x in X])

def _recorrer_arbol(self, x, nodo):
    if nodo.value is not None: return nodo.value
    if x[nodo.feature] <= nodo.threshold:
        return self._recorrer_arbol(x, nodo.left)
    return self._recorrer_arbol(x, nodo.right)

import networkx as nx
import matplotlib.pyplot as plt

def dibujar_estructura_arbol(nodo_raiz, feature_names):
    grafo = nx.DiGraph()
    etiquetas = {}

    def añadir_nodos(nodo, padre_id=None, pos="root", id_actual=0):
        if nodo is None: return id_actual

        # Texto del nodo
        if nodo.value is not None:
            texto = f"HOJA\nClase: {nodo.value}"
        else:
            texto = f"{feature_names[nodo.feature]}\n<= {nodo.threshold}"

        etiquetas[id_actual] = texto
        grafo.add_node(id_actual)

        if padre_id is not None:
            grafo.add_edge(padre_id, id_actual, label=pos)

        proximo_id = id_actual + 1
        if nodo.left:
            proximo_id = añadir_nodos(nodo.left, id_actual, "Si", proximo_id)
        if nodo.right:
            proximo_id = añadir_nodos(nodo.right, id_actual, "No",
proximo_id)

        return proximo_id

    añadir_nodos(nodo_raiz)

    # Layout simple para no depender de Graphviz externo
    posiciones = nx.spring_layout(grafo)

    plt.figure(figsize=(10, 6))
    nx.draw(grafo, posiciones, with_labels=False, node_size=3000,
node_color="skyblue", node_shape="s")
    nx.draw_networkx_labels(grafo, posiciones, labels=etiquetas, font_size=9)

    edge_labels = nx.get_edge_attributes(grafo, 'label')
    nx.draw_networkx_edge_labels(grafo, posiciones, edge_labels=edge_labels)

```

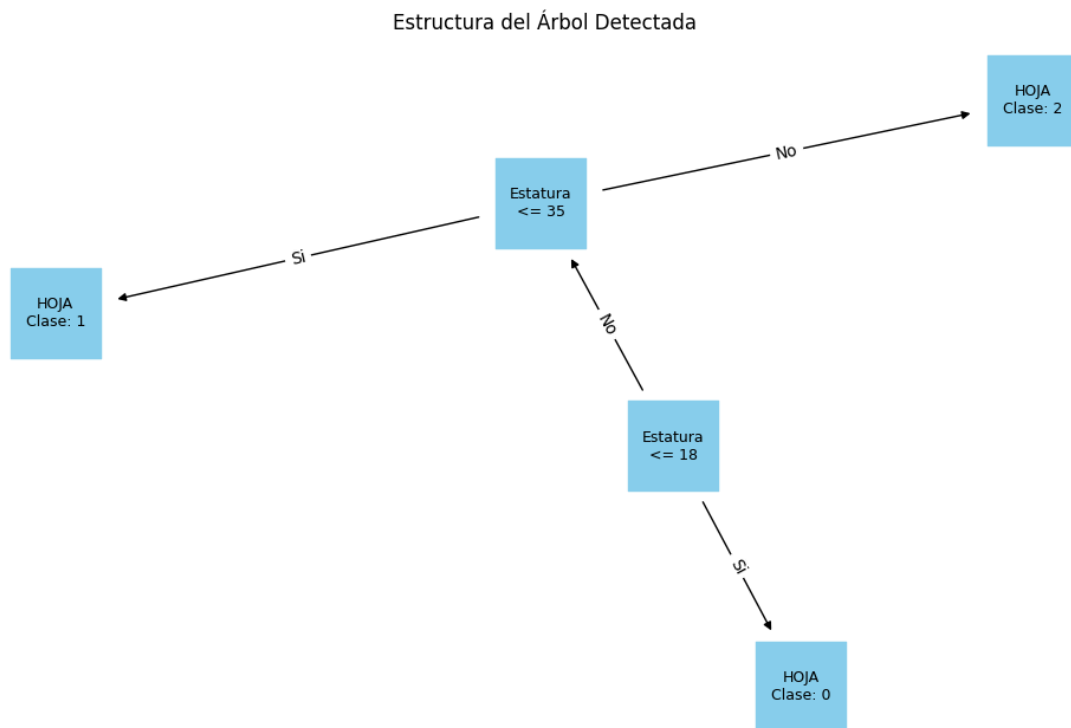


```
plt.title("Estructura del Árbol Detectada")
plt.show()
```

```
# --- Ejemplo de uso ---
if __name__ == "__main__":
    X_train = np.array([[15, 4], [18, 5], [20, 6], [35, 20], [40, 25], [45,
30]])
    y_train = np.array([0, 0, 1, 1, 2, 2])

    modelo = ArbolDecision(max_depth=2)
    modelo.fit(X_train, y_train)

    # Dibujamos usando la función externa pasándole la raíz
    dibujar_estructura_arbol(modelo.root, feature_names=["Estatura", "Peso"])
```



```
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt
import networkx as nx

# =====
# --- REUTILIZAMOS TU CLASE NODO ORIGINAL (SIN CAMBIOS) ---
# =====
class Nodo:
    def __init__(self, feature=None, threshold=None, left=None, right=None,
value=None):
        self.feature = feature
```

```

        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

# =====
# --- NUEVA FUNCIÓN DE DIBUJO CON POSICIONAMIENTO LÓGICO ---
# =====

def _calcular_posiciones(nodo, x=0.5, y=1.0, width=0.5, current_depth=0,
total_depth=1, node_positions={}, node_info={}):
    """
    Función recursiva para calcular las posiciones X e Y de un árbol
    balanceado.
    Calcula el espacio disponible para cada nodo y centra sus hijos.
    """
    if nodo is None:
        return node_positions, node_info

    # Asignar posición (X, Y)
    # X va de 0 a 1. Y va de 1 (raíz) a 0 (hojas más profundas).
    depth_factor = 1.0 / total_depth
    node_id = id(nodo) # Usamos el ID del objeto para NetworkX
    node_positions[node_id] = (x, 1.0 - (current_depth * depth_factor))

    # Guardar la información descriptiva del nodo (pregunta o clase)
    if nodo.value is not None:
        info_text = f"HOJA:\nClase: {nodo.value}"
    else:
        info_text = f"P: {nodo.feature} <= {nodo.threshold:.1f}" # P de
        Pregunta y formato .1f

    node_info[node_id] = {'label': info_text, 'type': 'leaf' if nodo.value is
not None else 'split'}

    # Recursión para hijos
    next_y = y - depth_factor # Reducir Y para el siguiente nivel
    new_width = width / 2.0    # El espacio disponible se divide a la mitad

    if nodo.left:
        # Hijo izquierdo va a la izquierda (- ancho/2)
        _calcular_posiciones(nodo.left, x - new_width, next_y, new_width,
current_depth + 1, total_depth, node_positions, node_info)

    if nodo.right:
        # Hijo derecho va a la derecha (+ ancho/2)
        _calcular_posiciones(nodo.right, x + new_width, next_y, new_width,
current_depth + 1, total_depth, node_positions, node_info)

    return node_positions, node_info

def obtener_max_depth(nodo):
    """Calcula la profundidad máxima real del árbol."""

```

```

    if nodo is None or nodo.value is not None:
        return 0
    return 1 + max(obtener_max_depth(nodo.left),
obtener_max_depth(nodo.right))

def dibujar_estructura_balanceada(nodo_raiz, feature_names, title="Estructura
del Árbol Detectada"):
    """
    Función principal de visualización jerárquica y balanceada.
    Calcula las posiciones jerárquicas y dibuja el árbol.
    """
    if nodo_raiz is None:
        print("El árbol no ha sido entrenado o está vacío.")
        return

    # 1. Crear el grafo NetworkX y calcular posiciones lógicas
    grafo = nx.DiGraph()
    depth_real = obtener_max_depth(nodo_raiz)
    # depth + 2 para que las hojas no queden pegadas al borde inferior
    node_pos, node_data = _calcular_posiciones(nodo_raiz,
total_depth=depth_real + 2)

    # 2. Re-etiquetar con nombres de características
    etiquetas_texto = {}
    for nid, data in node_data.items():
        if data['type'] == 'split':
            # Reemplazar P:0 con Estatura:
            text = data['label']
            p_idx = text.split(":")[1].split("<")[0].strip() # Obtener el
índice 0
            new_text = text.replace(f"P: {p_idx}", feature_names[int(p_idx)])
            etiquetas_texto[nid] = new_text
        else:
            etiquetas_texto[nid] = data['label']

    # 3. Añadir nodos y aristas al grafo de NetworkX
    grafo.add_nodes_from(node_pos.keys())

    def añadir_aristas(nodo, graph):
        if nodo is None or nodo.value is not None: return
        if nodo.left:
            graph.add_edge(id(nodo), id(nodo.left), label="Si")
            añadir_aristas(nodo.left, graph)
        if nodo.right:
            graph.add_edge(id(nodo), id(nodo.right), label="No")
            añadir_aristas(nodo.right, graph)

    añadir_aristas(nodo_raiz, grafo)

    # 4. Dibujar el árbol con Matplotlib
    plt.figure(figsize=(10, 7))
    plt.title(title, fontsize=16)

```

```

# Dibujar nodos cuadrados y celestes, como en tu imagen
nx.draw(grafo, node_pos, with_labels=False, node_size=3000,
        node_color="#87CEEB", node_shape='s', edge_color='black',
arrows=True)

# Dibujar etiquetas dentro de los nodos
nx.draw_networkx_labels(grafo, node_pos, labels=etiquetas_texto,
font_size=9)

# Dibujar etiquetas de las aristas (Si/No)
edge_labels = nx.get_edge_attributes(grafo, 'label')
nx.draw_networkx_edge_labels(grafo, node_pos, edge_labels=edge_labels,
font_size=10, font_color='black')

# Configuración de ejes para que el layout se vea perfecto
plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)
plt.axis('off') # Ocultar ejes de coordenadas
plt.tight_layout()
plt.show()

# =====
# --- BLOQUE PRINCIPAL DE EJECUCIÓN ---
# =====
if __name__ == "__main__":
    # 1. Dataset simple (Gatos vs Perros) [Estatura, Peso]
    X_train = np.array([[15, 4], [18, 5], [20, 6], [35, 20], [40, 25], [45,
30]])
    y_train = np.array([0, 0, 0, 1, 1, 1]) # 0: Gato, 1: Perro

    # 2. Entrenar el árbol
    # YA NO USAMOS 'from solution import...', usamos la clase que definimos
arriba
    modelo = ArbolDecision(max_depth=3)
    modelo.fit(X_train, y_train)

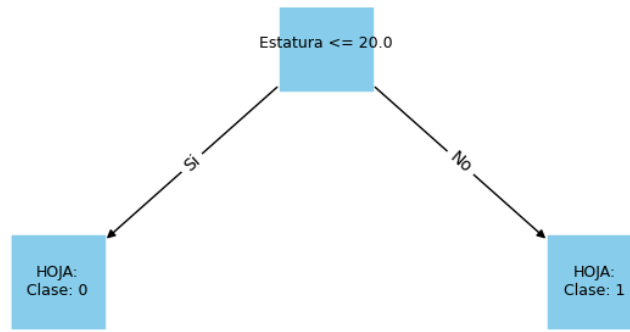
    # 3. Clasificar un punto nuevo
    X_nuevo = np.array([[25, 10]])
    print(f"Predicción para {X_nuevo[0]}: Clase
{modelo.predict(X_nuevo)[0]}")

    # 4. DIBUJAR EL ÁRBOL BALANCEADO
    # Usamos la función que creamos para que se vea ordenado
    dibujar_estructura_balanceada(modelo.root, feature_names=["Estatura",
"Peso"])

```

Predicción para [25 10]: Clase 1

Estructura del Árbol Detectada



Random Forest

Aquí reutilizamos tu clase `ArbolDecision` (asegúrate de tenerla en el mismo script o importada).

```
import numpy as np
from collections import Counter

class RandomForest:
    """
    Random Forest: ensemble de árboles con Bootstrap Aggregating (Bagging).
    Predicción por votación mayoritaria:  $\hat{y} = \operatorname{argmax}_c \sum 1[h_i(x)=c]$ 

    Args:
        n_trees (int): Número de árboles.
        max_depth (int): Profundidad máxima de cada árbol.
    """

    def __init__(self, n_trees=10, max_depth=5):
        self.n_trees = n_trees
        self.max_depth = max_depth
        self.trees = []

    def fit(self, X, y):
        """
        Entrena n_trees árboles sobre muestras bootstrap.
        """
```

```

Args:
    X (np.ndarray): Shape (m, n).
    y (np.ndarray): Shape (m,).

Returns:
    self
"""
try:
    if X.shape[0] != y.shape[0]:
        raise ValueError("X e y tienen dimensiones incompatibles.")
    self.trees = []
    for _ in range(self.n_trees):
        idx = np.random.choice(X.shape[0], X.shape[0], replace=True)
        tree = ArbolDecision(max_depth=self.max_depth)
        tree.fit(X[idx], y[idx])
        self.trees.append(tree)
    logger.info(
        "RandomForest: %d árboles entrenados.", self.n_trees
    )
    return self
except Exception as e:
    logger.error("Error en RandomForest.fit: %s", e)
    raise

def predict(self, X):
    """
    Predice por votación mayoritaria de todos los árboles.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Etiquetas predichas.
    """
    try:
        if not self.trees:
            raise RuntimeError("Modelo no entrenado.")
        preds = np.swapaxes(
            np.array([t.predict(X) for t in self.trees]), 0, 1
        )
        return np.array(
            [Counter(p).most_common(1)[0][0] for p in preds]
        )
    except Exception as e:
        logger.error("Error en RandomForest.predict: %s", e)
        raise

```

Visualización Independiente Como Random Forest son muchos árboles, lo más didáctico es dibujar una "galería" de los primeros árboles para que vean que efectivamente son diferentes entre sí.

```

def visualizar_bosque_completo(forest, feature_names, n_a_mostrar=3):
    """
    Muestra una hilera de árboles del bosque para comparar sus estructuras.
    """
    fig = plt.figure(figsize=(n_a_mostrar * 5, 5))

    for i in range(n_a_mostrar):
        # Crear un sub-eje para cada árbol
        ax = fig.add_subplot(1, n_a_mostrar, i + 1)

        # Obtenemos la raíz del árbol i-ésimo
        raiz_actual = forest.trees[i].root

        # Reutilizamos la lógica de cálculo de posiciones
        # Nota: He ajustado la función dibujar para que acepte un 'ax'
        # específico
        dibujar_en_subfigura(raiz_actual, feature_names, ax, f"Árbol #{i+1}")

    plt.tight_layout()
    plt.show()

def dibujar_en_subfigura(nodo_raiz, feature_names, ax, titulo):
    """Versión simplificada de la función de dibujo para cuadrículas."""
    grafo = nx.DiGraph()
    depth_real = obtener_max_depth(nodo_raiz)
    node_pos, node_data = _calcular_posiciones(nodo_raiz,
    total_depth=depth_real + 2)

    # Construir etiquetas y aristas (mismo proceso que antes)
    etiquetas_texto = {nid: d['label'] for nid, d in node_data.items()}

    def añadir_aristas(nodo, graph):
        if nodo is None or nodo.value is not None: return
        if nodo.left:
            graph.add_edge(id(nodo), id(nodo.left), label="S")
            añadir_aristas(nodo.left, graph)
        if nodo.right:
            graph.add_edge(id(nodo), id(nodo.right), label="N")
            añadir_aristas(nodo.right, graph)

    añadir_aristas(nodo_raiz, grafo)

    # Dibujar en el eje 'ax' proporcionado
    nx.draw(grafo, node_pos, ax=ax, with_labels=False, node_size=1500,
            node_color="#87CEEB", node_shape='s', edge_color='gray',
    arrows=True)
    nx.draw_networkx_labels(grafo, node_pos, ax=ax, labels=etiquetas_texto,
    font_size=7)
    ax.set_title(titulo)
    ax.axis('off')

# Entrenamos el bosque

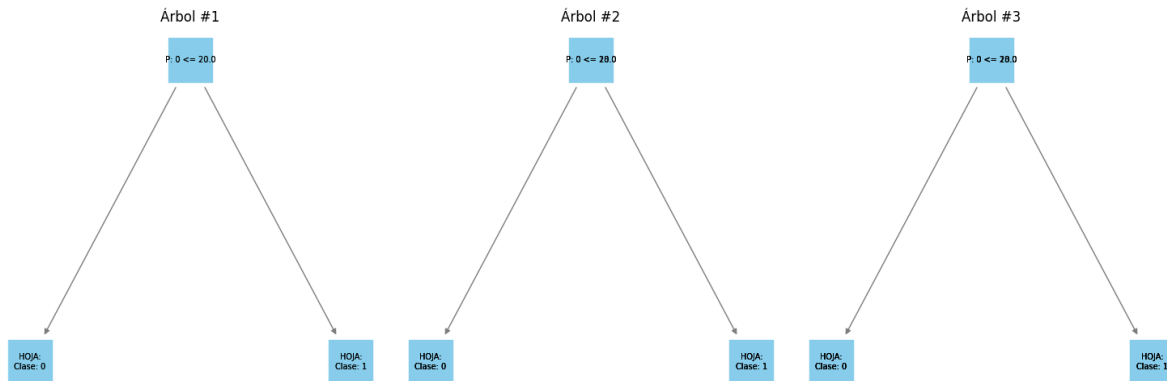
```

```

bosque = RandomForest(n_trees=10, max_depth=3)
bosque.fit(X_train, y_train)

# Visualizamos los primeros 3 para no saturar la pantalla
visualizar_bosque_completo(bosque, ["Estatura", "Peso"], n_a_mostrar=3)

```



Gradient Boosting

Para que el Gradient Boosting funcione, nuestro ArbolDecision debe ser capaz de manejar valores continuos. Se ha simplificado la lógica del árbol para que use la Varianza como medida de impureza (que es el equivalente al MSE).

Ahora estamos haciendo regresión sobre los residuos, debemos cambiar la métrica del árbol. Ya no usaremos Entropía (que es para clasificación), sino el Error Cuadrático Medio (MSE).

```

import numpy as np
from collections import Counter

class Nodo:
    """Nodo de un árbol de decisión binario.

    Puede ser un nodo interno (pregunta) o una hoja (valor).

    Args:
        feature (int | None) : Índice de la característica de corte.
        threshold (float | None): Valor umbral del corte.
        left (Nodo | None) : Sub-árbol izquierdo (<= threshold).
        right (Nodo | None) : Sub-árbol derecho (> threshold).
        value (float | None): Predicción de la hoja (si es hoja).
    """

    def __init__(
        self,
        feature: int | None = None,
        threshold: float | None = None,
        left: "Nodo | None" = None,
        right: "Nodo | None" = None,
    ):

```



```

        value: float | None = None,
    ) -> None:
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

class ArbolDecisionRegresor:
    """
    Árbol de Decisión para regresión (valores continuos).
    Usa varianza ponderada (MSE) como criterio de corte:
         $MSE = (nL \cdot \text{var}(yL) + nR \cdot \text{var}(yR)) / n$ 

    Args:
        max_depth (int): Profundidad máxima.
    """

    def __init__(self, max_depth=3):
        self.max_depth = max_depth
        self.root = None

    def fit(self, X, y):
        """
        Construye el árbol minimizando MSE en cada corte.

        Args:
            X (np.ndarray): Shape (m, n).
            y (np.ndarray): Valores continuos, shape (m,).

        Returns:
            self
        """
        try:
            self.root = self._crear_arbol(X, y, depth=0)
            return self
        except Exception as e:
            logger.error("Error en ArbolDecisionRegresor.fit: %s", e)
            raise

    def _crear_arbol(self, X, y, depth):
        """Hoja = promedio de y cuando se alcanza condición de parada."""
        if depth >= self.max_depth or len(y) < 2:
            return Nodo(value=np.mean(y))
        best_feat, best_thresh = self._mejor_criterio(X, y)
        if best_feat is None:
            return Nodo(value=np.mean(y))
        left_idx = X[:, best_feat] <= best_thresh
        izq = self._crear_arbol(X[left_idx], y[left_idx], depth + 1)
        der = self._crear_arbol(X[~left_idx], y[~left_idx], depth + 1)
        return Nodo(
            feature=best_feat, threshold=best_thresh,
            left=izq, right=der
        )

```

```

    )

def _mejor_criterio(self, X, y):
    """Busca el corte que minimiza la varianza ponderada."""
    best_mse = np.var(y)
    split_idx, split_thresh = None, None
    for feat_idx in range(X.shape[1]):
        for threshold in np.unique(X[:, feat_idx]):
            ly = y[X[:, feat_idx] <= threshold]
            ry = y[X[:, feat_idx] > threshold]
            if len(ly) == 0 or len(ry) == 0:
                continue
            # MSE ponderado
            mse = (len(ly) * np.var(ly) + len(ry) * np.var(ry)) / len(y)
            if mse < best_mse:
                best_mse = mse
                split_idx = feat_idx
                split_thresh = threshold
    return split_idx, split_thresh

def predict(self, X):
    """
    Predice valores continuos recorriendo el árbol.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Valores predichos.
    """
    try:
        if self.root is None:
            raise RuntimeError("Modelo no entrenado.")
        return np.array([self._recorrer(x, self.root) for x in X])
    except Exception as e:
        logger.error("Error en ArbolDecisionRegresor.predict: %s", e)
        raise

def _recorrer(self, x, nodo):
    """Recorre hasta hoja retornando el promedio almacenado."""
    if nodo.value is not None:
        return nodo.value
    if x[nodo.feature] <= nodo.threshold:
        return self._recorrer(x, nodo.left)
    return self._recorrer(x, nodo.right)

#
-----
--

class GradientBoostingDesdeCero:
    """

```

Gradient Boosting: ensemble aditivo de árboles de regresión.
En cada iteración entrena sobre los residuos:

$$\text{residuos} = y - F_m(x)$$
$$F_{\{m+1\}}(x) = F_m(x) + lr * h_m(x)$$

Args:

n_estimators (int): Número de árboles.
learning_rate (float): Peso de cada árbol (lr).
max_depth (int): Profundidad máxima de cada árbol.

"""

```
def __init__(self, n_estimators=10, learning_rate=0.1, max_depth=3):
    self.n_estimators = n_estimators
    self.learning_rate = learning_rate
    self.max_depth = max_depth
    self.trees = []
    self.init_prediction = None
```

```
def fit(self, X, y):
```

"""

Entrena el ensemble ajustando residuos iterativamente.

Args:

X (np.ndarray): Shape (m, n).
y (np.ndarray): Valores objetivo, shape (m,).

Returns:

self

"""

try:

```
    y = y.astype(float)
    # Predicción inicial: promedio de y
    self.init_prediction = np.mean(y)
    f_m = np.full(y.shape, self.init_prediction)

    for _ in range(self.n_estimators):
        residuos = y - f_m  # gradiente negativo
        tree = ArbolDecisionRegresor(max_depth=self.max_depth)
        tree.fit(X, residuos)
        f_m += self.learning_rate * tree.predict(X)
        self.trees.append(tree)
```

```
    logger.info(
        "GradientBoosting: %d estimadores entrenados.",
        self.n_estimators
    )
```

return self

except Exception as e:

```
    logger.error("Error en GradientBoosting.fit: %s", e)
    raise
```

```
def predict(self, X):
```

"""

```

        Suma la contribución de todos los árboles.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Predicciones continuas.
    """
    try:
        if not self.trees:
            raise RuntimeError("Modelo no entrenado.")
        y_pred = np.full(X.shape[0], self.init_prediction)
        for tree in self.trees:
            y_pred += self.learning_rate * tree.predict(X)
        return y_pred
    except Exception as e:
        logger.error("Error en GradientBoosting.predict: %s", e)
        raise

#
-----
--

class ArbolRegresorRegularizado(ArbolDecisionRegresor):
    """
    Árbol regresor con hoja regularizada al estilo XGBoost.
    Valor de hoja:  $w^* = \Sigma g / (\Sigma h + \lambda)$ 
    Para MSE:  $g = \text{residuos}$ ,  $h = 1 \rightarrow w^* = \Sigma \text{resid} / (n + \lambda)$ 

    Args:
        max_depth (int): Profundidad máxima.
        reg_lambda (float): Término de regularización L2.
    """

    def __init__(self, max_depth=3, reg_lambda=1.0):
        super().__init__(max_depth)
        self.reg_lambda = reg_lambda

    def _crear_arbol(self, X, y, depth):
        """Hoja con peso regularizado:  $\Sigma y / (n + \lambda)$ ."""
        if depth >= self.max_depth or len(y) < 2:
            peso = np.sum(y) / (len(y) + self.reg_lambda)
            return Nodo(value=peso)
        return super()._crear_arbol(X, y, depth)

# --- Prueba sin Errores ---
if __name__ == "__main__":
    X = np.array([[1], [2], [3], [4], [5]])
    y = np.array([2, 4, 6, 8, 10])

    gb = GradientBoostingDesdeCero(n_estimators=50, learning_rate=0.1)
    gb.fit(X, y)

```

```
test_val = np.array([[6]])
print(f"Predicción para x=6: {gb.predict(test_val)[0]:.2f}")
```

Predicción para x=6: 9.98

XGBoost

Implementaremos la Regularización L2 () sobre nuestro regresor de residuos, que es el alma de XGBoost.

```
import numpy as np
```

```
class XGBoostMiniDesdeCero:
    """
    XGBoost simplificado: Gradient Boosting con regularización L2.
    Reduce el sobreajuste penalizando la magnitud de las hojas:
         $w^* = \Sigma g / (\Sigma h + \lambda)$ 

    Args:
        n_estimators (int): Número de árboles.
        learning_rate (float): Tasa de aprendizaje.
        max_depth (int): Profundidad máxima.
        reg_lambda (float): Regularización L2.
    """

    def __init__(
        self, n_estimators=10, learning_rate=0.1,
        max_depth=3, reg_lambda=1.0
    ):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.reg_lambda = reg_lambda
        self.trees = []
        self.init_prediction = 0

    def fit(self, X, y):
        """
        Entrena con árboles regularizados sobre residuos.

        Args:
            X (np.ndarray): Shape (m, n).
            y (np.ndarray): Valores objetivo, shape (m,).

        Returns:
            self
        """
```

```

try:
    self.init_prediction = np.mean(y)
    f_m = np.full(y.shape, self.init_prediction)

    for _ in range(self.n_estimators):
        residuos = y - f_m
        tree = ArbolRegresorRegularizado(
            max_depth=self.max_depth,
            reg_lambda=self.reg_lambda
        )
        tree.fit(X, residuos)
        f_m += self.learning_rate * tree.predict(X)
        self.trees.append(tree)

    logger.info("XGBoostMini: %d estimadores.", self.n_estimators)
    return self
except Exception as e:
    logger.error("Error en XGBoostMini.fit: %s", e)
    raise

def predict(self, X):
    """
    Predice sumando la contribución regularizada de cada árbol.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Predicciones continuas.
    """
    try:
        if not self.trees:
            raise RuntimeError("Modelo no entrenado.")
        y_pred = np.full(X.shape[0], self.init_prediction)
        for tree in self.trees:
            y_pred += self.learning_rate * tree.predict(X)
        return y_pred
    except Exception as e:
        logger.error("Error en XGBoostMini.predict: %s", e)
        raise

import matplotlib.pyplot as plt

def probar_xgboost_vs_realidad():
    # 1. Crear datos: Una curva seno con un poco de "ruido"
    # Esto simula datos reales que no son perfectamente lineales
    np.random.seed(42)
    X = np.sort(5 * np.random.rand(80, 1), axis=0)
    y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

    # 2. Instanciar y entrenar nuestro XGBoost "Mini"

```

```

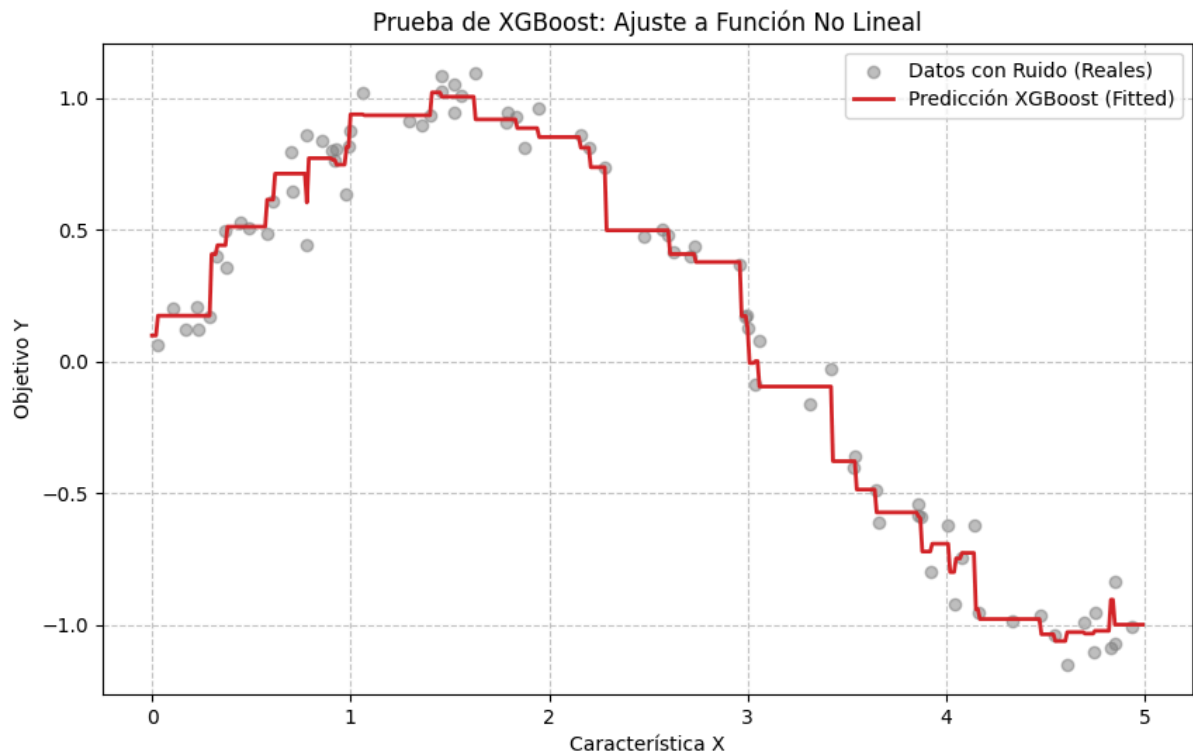
# Prueba variando reg_lambda: si es 0, se sobreajusta; si es alto, es
más suave
modelo = XGBoostMiniDesdeCero(n_estimators=30, learning_rate=0.2,
max_depth=3, reg_lambda=1.0)
modelo.fit(X, y)

# 3. Generar predicciones para visualizar la curva
X_test = np.arange(0.0, 5.0, 0.01)[: , np.newaxis]
y_pred = modelo.predict(X_test)

# 4. Graficar los resultados
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color="tab:gray", alpha=0.5, label="Datos con Ruido
(Reales)")
plt.plot(X_test, y_pred, color="tab:red", linewidth=2, label="Predicción
XGBoost (Fitted)")
plt.title("Prueba de XGBoost: Ajuste a Función No Lineal")
plt.xlabel("Característica X")
plt.ylabel("Objetivo Y")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

if __name__ == "__main__":
    probar_xgboost_vs_realidad()

```



K-means

```

import numpy as np
import matplotlib.pyplot as plt

class KMeans:
    """
    K-Means: minimiza la inercia  $J = \sum ||x_i - \mu_c||^2$ 
    Itera: (1) asignar al centroide más cercano, (2) recalcular centroides.

    Args:
        K (int): Número de clusters.
        max_iters (int): Máximo de iteraciones.
    """

    def __init__(self, K=3, max_iters=100):
        self.K = K
        self.max_iters = max_iters
        self.centroids = None
        self.clusters = None

    def fit(self, X):
        """
        Ejecuta K-Means hasta convergencia.

        Args:
            X (np.ndarray): Datos a agrupar, shape (m, n).

        Returns:
            self
        """
        try:
            if self.K > X.shape[0]:
                raise ValueError("K no puede ser mayor que m.")
            self.X = X
            n_samples, n_features = X.shape
            idx = np.random.choice(n_samples, self.K, replace=False)
            self.centroids = X[idx].copy()

            for it in range(self.max_iters):
                self.clusters = self._assign(self.centroids)
                old = self.centroids.copy()
                self.centroids = self._update(self.clusters, n_features)
                if np.allclose(old, self.centroids):
                    logger.info(
                        "KMeans convergió en iteración %d.", it + 1
                    )
                    break
            return self
        except Exception as e:
            logger.error("Error en KMeans.fit: %s", e)
            raise

    def _assign(self, centroids):

```



```

        """Asigna cada punto al centroide más cercano."""
        clusters = [[] for _ in range(self.K)]
        for idx, x in enumerate(self.X):
            dists = [np.sqrt(np.sum((x - c) ** 2)) for c in centroids]
            clusters[np.argmin(dists)].append(idx)
        return clusters

def _update(self, clusters, n_features):
    """Recalcula centroides:  $\mu_i = (1/|C_i|) \sum x_i$ ."""
    centroids = np.zeros((self.K, n_features))
    for j, cluster in enumerate(clusters):
        if cluster:
            centroids[j] = np.mean(self.X[cluster], axis=0)
        else:
            centroids[j] = self.X[np.random.randint(len(self.X))]
    return centroids

def predict(self, X):
    """
    Asigna nuevas muestras al centroide más cercano.

    Args:
        X (np.ndarray): Shape (m, n).

    Returns:
        np.ndarray: Índice de cluster por muestra.
    """
    try:
        if self.centroids is None:
            raise RuntimeError("Modelo no entrenado.")
        dists = [
            np.sqrt(np.sum((X - c) ** 2, axis=1))
            for c in self.centroids
        ]
        return np.argmin(dists, axis=0)
    except Exception as e:
        logger.error("Error en KMeans.predict: %s", e)
        raise

# --- Prueba para el laboratorio ---
if __name__ == "__main__":
    # Crear datos sintéticos (3 grupos claros)
    from sklearn.datasets import make_blobs
    X, _ = make_blobs(n_samples=300, centers=3, cluster_std=0.60,
                      random_state=0)

    # Entrenar modelo
    km = KMeans(K=3)
    km.fit(X)

    # Graficar
    plt.figure(figsize=(8, 6))

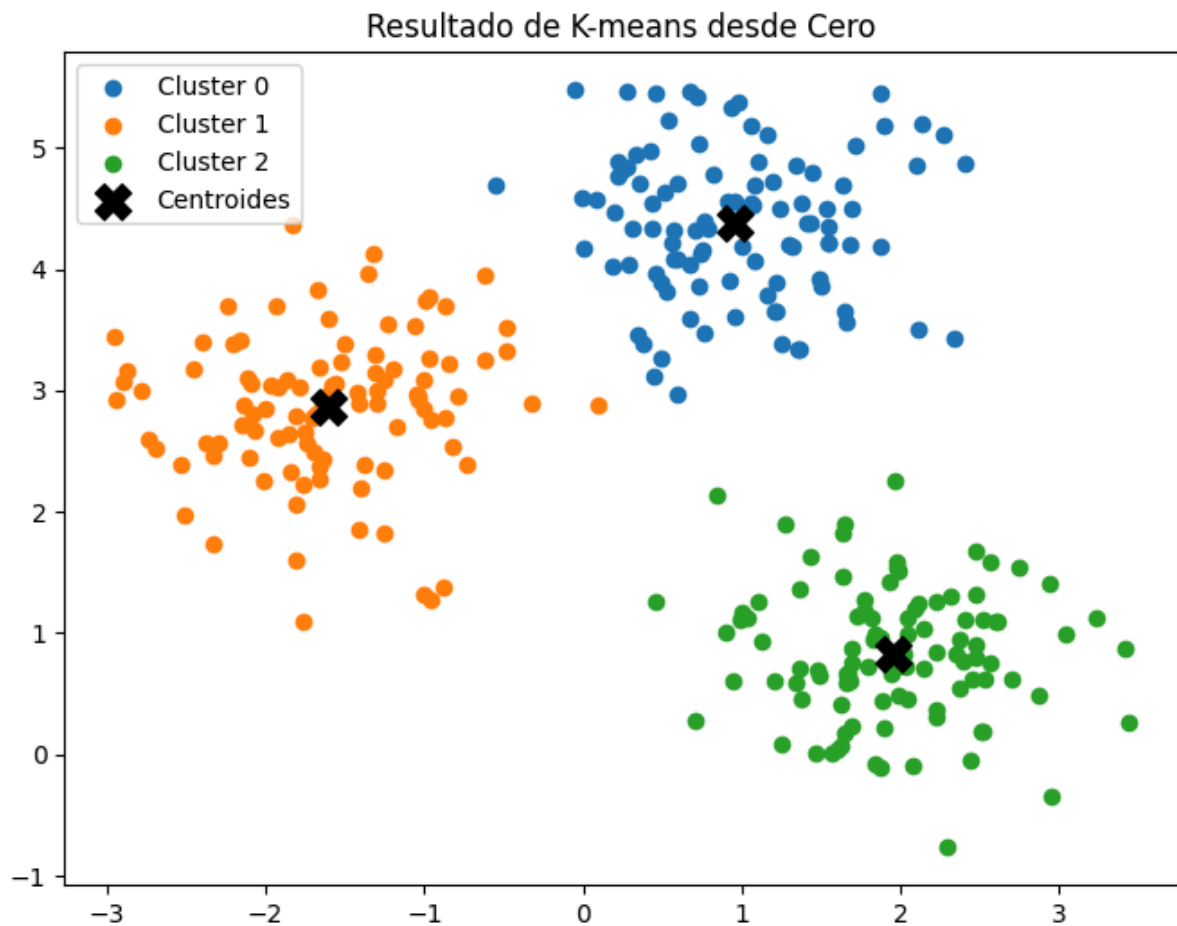
```

```

for i, cluster in enumerate(km.clusters):
    points = X[cluster]
    plt.scatter(points[:, 0], points[:, 1], label=f"Cluster {i}")

    # Dibujar los centroides finales
    plt.scatter(km.centroids[:, 0], km.centroids[:, 1], s=200, c='black',
marker='X', label="Centroides")
plt.title("Resultado de K-means desde Cero")
plt.legend()
plt.show()

```



DBSCAN

```

import numpy as np
import matplotlib.pyplot as plt

class DBSCAN:
    """
    DBSCAN: clustering por densidad, detecta ruido (label = -1).
    Punto núcleo: |Vecindad_ε(p)| ≥ min_samples

    Args:

```

```

        eps (float): Radio de vecindad  $\epsilon$ .
        min_samples (int): Mínimo de puntos para ser núcleo.
    """

    def __init__(self, eps=0.5, min_samples=5):
        self.eps = eps
        self.min_samples = min_samples
        self.labels = None

    def fit(self, X):
        """
        Agrupa X detectando núcleos, bordes y ruido.

        Args:
            X (np.ndarray): Shape (m, n).

        Returns:
            np.ndarray: Etiquetas de cluster (-1 = ruido), shape (m,).
        """
        try:
            n = X.shape[0]
            self.labels = np.full(n, -1)
            visited = np.zeros(n, dtype=bool)
            cluster_id = 0

            for i in range(n):
                if visited[i]:
                    continue
                visited[i] = True
                neighbors = self._neighbors(X, i)
                if len(neighbors) < self.min_samples:
                    self.labels[i] = -1
                else:
                    self._expand(X, i, neighbors, cluster_id, visited)
                    cluster_id += 1

            logger.info(
                "DBSCAN: %d clusters, %d ruido.",
                cluster_id, np.sum(self.labels == -1)
            )
            return self.labels
        except Exception as e:
            logger.error("Error en DBSCAN.fit: %s", e)
            raise

    def _neighbors(self, X, i):
        """Retorna índices de puntos dentro del radio  $\epsilon$  de X[i]."""
        return np.where(
            np.linalg.norm(X - X[i], axis=1) <= self.eps
        )[0]

    def _expand(self, X, root, neighbors, cid, visited):
        """BFS para expandir el cluster desde el punto núcleo root."""

```

```

        self.labels[root] = cid
        queue = list(neighbors)
        while queue:
            p = queue.pop(0)
            if not visited[p]:
                visited[p] = True
                nb = self._neighbors(X, p)
                if len(nb) >= self.min_samples:
                    queue.extend(nb)
            if self.labels[p] == -1:
                self.labels[p] = cid

# --- Prueba para el laboratorio (Formas no circulares) ---
if __name__ == "__main__":
    from sklearn.datasets import make_moons
    # Creamos dos lunas (K-means fallaría aquí)
    X, _ = make_moons(n_samples=200, noise=0.05, random_state=0)

    # Entrenar DBSCAN
    db = DBSCAN(eps=0.2, min_samples=5)
    labels = db.fit(X)

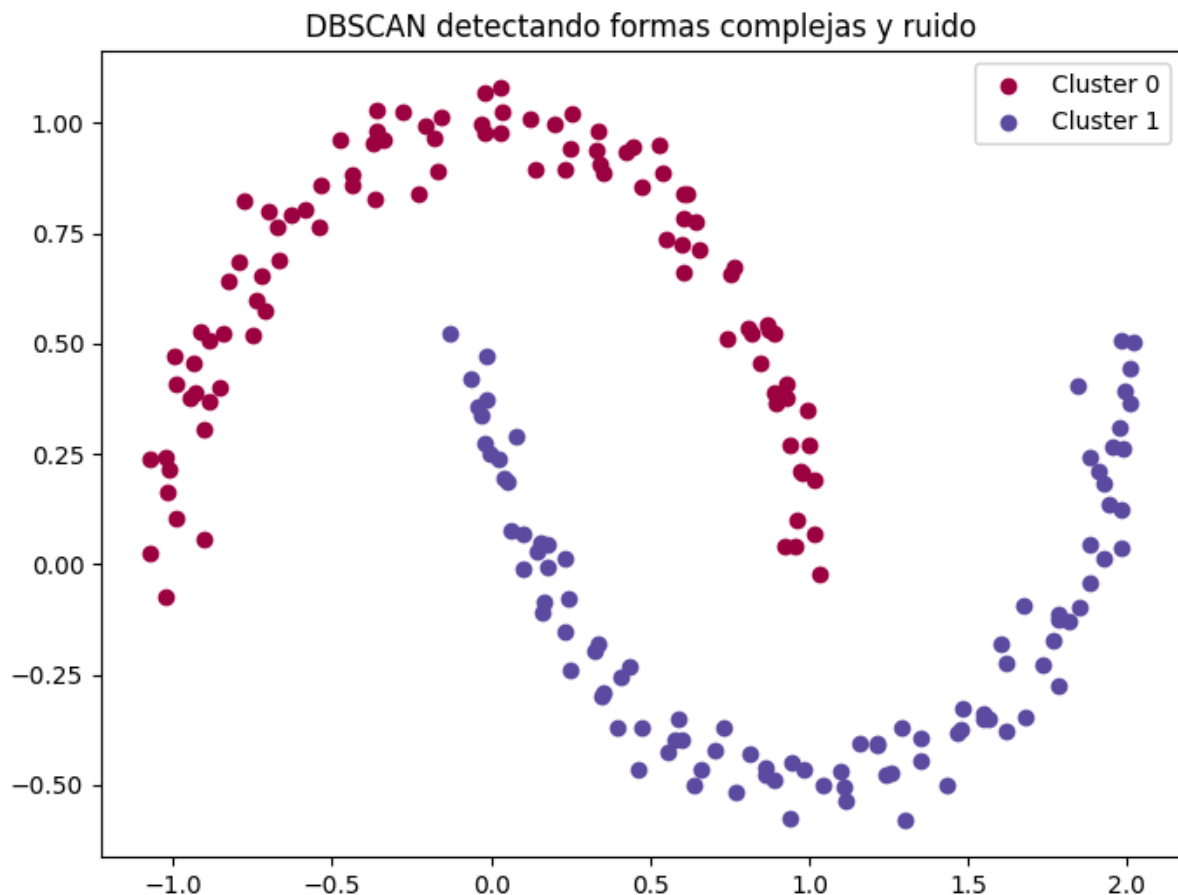
    # Graficar
    plt.figure(figsize=(8, 6))
    unique_labels = np.unique(labels)
    colors = plt.cm.Spectral(np.linspace(0, 1, len(unique_labels)))

    for k, col in zip(unique_labels, colors):
        if k == -1:
            col = 'black' # Ruido en negro

        class_member_mask = (labels == k)
        plt.scatter(X[class_member_mask, 0], X[class_member_mask, 1],
                    c=[col], label=f"Cluster {k}" if k != -1 else "Ruido")

    plt.title("DBSCAN detectando formas complejas y ruido")
    plt.legend()
    plt.show()

```



Hierarchical Clustering

```
import numpy as np
import matplotlib.pyplot as plt

class HierarchicalClustering:
    """
    Clustering Jerárquico Aglomerativo (Single Linkage).
    Fusiona iterativamente los dos clusters más cercanos hasta
    llegar a n_clusters. Distancia entre clusters:
         $d(C1, C2) = \min\{||x_i - x_j|| : x_i \in C1, x_j \in C2\}$ 

    Args:
        n_clusters (int): Número de clusters finales.
    """

    def __init__(self, n_clusters=2):
        self.n_clusters = n_clusters
        self.labels_ = None

    def fit(self, X):
        """
        Agrupa X mediante fusiones sucesivas (bottom-up).
        """
```

```

Args:
    X (np.ndarray): Datos a agrupar, shape (m, n).

Returns:
    np.ndarray: Etiquetas de cluster, shape (m,).
"""
try:
    # Cada punto empieza como su propio cluster
    clusters = [[i] for i in range(len(X))]

    while len(clusters) > self.n_clusters:
        min_dist = float('inf')
        pair = (0, 1)

        # Buscar el par de clusters más cercanos
        for i in range(len(clusters)):
            for j in range(i + 1, len(clusters)):
                d = self._dist_clusters(X, clusters[i], clusters[j])
                if d < min_dist:
                    min_dist = d
                    pair = (i, j)

        # Fusionar
        c1, c2 = pair
        clusters.append(clusters[c1] + clusters[c2])
        clusters.pop(c2)
        clusters.pop(c1)

    self.labels_ = self._get_labels(len(X), clusters)
    logger.info(
        "HierarchicalClustering: %d clusters finales.",
        self.n_clusters
    )
    return self.labels_
except Exception as e:
    logger.error("Error en HierarchicalClustering.fit: %s", e)
    raise

def _dist_clusters(self, X, c1, c2):
    """Single linkage: mínima distancia entre puntos de c1 y c2."""
    return min(
        np.linalg.norm(X[i] - X[j])
        for i in c1 for j in c2
    )

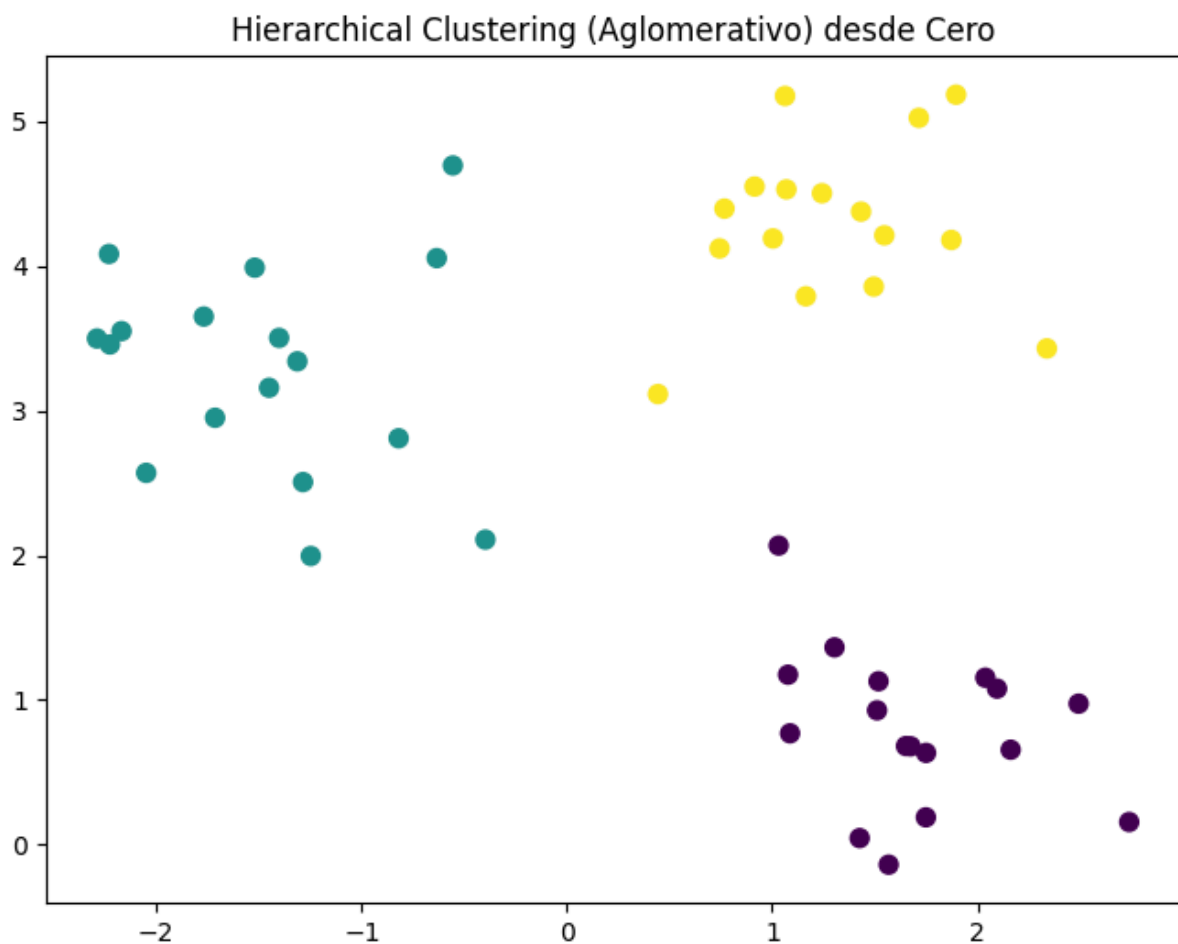
def _get_labels(self, n_samples, clusters):
    """Convierte lista de clusters a array de etiquetas."""
    labels = np.zeros(n_samples, dtype=int)
    for cid, cluster in enumerate(clusters):
        for point in cluster:
            labels[point] = cid
    return labels

```

```
# --- Prueba para el laboratorio ---
if __name__ == "__main__":
    from sklearn.datasets import make_blobs
    X, _ = make_blobs(n_samples=50, centers=3, cluster_std=0.6,
random_state=0)

    # Entrenar modelo
    hc = HierarchicalClustering(n_clusters=3)
    labels = hc.fit(X)

    # Graficar
    plt.figure(figsize=(8, 6))
    plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50)
    plt.title("Hierarchical Clustering (Aglomerativo) desde Cero")
    plt.show()
```



Isolation Forest

```
import numpy as np
```

```

class IsolationForest:
    """
    Isolation Forest: detección de anomalías por aislamiento.
    Las anomalías se aíslan en menos pasos que los puntos normales.
    Score:  $s(x) = 2^{(-E[h(x)] / c(n))}$ 
    donde  $c(n) = 2(\ln(n-1) + 0.5772) - 2(n-1)/n$ 

    Args:
        n_trees (int): Número de árboles de aislamiento.
        sample_size (int): Tamaño de submuestra por árbol.
    """

    def __init__(self, n_trees=100, sample_size=256):
        self.n_trees = n_trees
        self.sample_size = sample_size
        self.trees = []
        self.limit = int(np.ceil(np.log2(sample_size)))

    def fit(self, X):
        """
        Construye n_trees árboles de aislamiento sobre submuestras.

        Args:
            X (np.ndarray): Datos, shape (m, n).

        Returns:
            self
        """
        try:
            self.trees = []
            for _ in range(self.n_trees):
                idx = np.random.choice(
                    len(X), self.sample_size, replace=False
                )
                self.trees.append(self._build_tree(X[idx], 0))
            logger.info(
                "IsolationForest: %d árboles contruidos.", self.n_trees
            )
            return self
        except Exception as e:
            logger.error("Error en IsolationForest.fit: %s", e)
            raise

    def _build_tree(self, X, depth):
        """
        Construye un iTree recursivo particionando aleatoriamente.

        Args:
            X (np.ndarray): Submuestra actual.
            depth (int): Profundidad actual.

        Returns:
            dict: Nodo del árbol {'type', 'feature', 'threshold',

```



```

        'left', 'right', 'size'}.

"""
n = len(X)
if depth >= self.limit or n <= 1:
    return {'type': 'leaf', 'size': n}

feat = np.random.randint(0, X.shape[1])
lo, hi = X[:, feat].min(), X[:, feat].max()
if lo == hi:
    return {'type': 'leaf', 'size': n}

threshold = np.random.uniform(lo, hi)
left_idx = X[:, feat] < threshold
return {
    'type': 'split',
    'feature': feat,
    'threshold': threshold,
    'left': self._build_tree(X[left_idx], depth + 1),
    'right': self._build_tree(X[~left_idx], depth + 1),
}

def _path_length(self, x, node, depth):
    """Longitud de camino de x en el árbol (+ ajuste c para hojas)."""
    if node['type'] == 'leaf':
        size = node['size']
        return depth + (self._c(size) if size > 1 else 0)
    if x[node['feature']] < node['threshold']:
        return self._path_length(x, node['left'], depth + 1)
    return self._path_length(x, node['right'], depth + 1)

def _c(self, n):
    """Factor de normalización  $c(n) = 2(\ln(n-1)+0.5772) - 2(n-1)/n$ ."""
    if n <= 1:
        return 0
    return 2.0 * (np.log(n - 1) + 0.5772156649) - 2.0 * (n - 1) / n

def anomaly_score(self, X):
    """
    Calcula el score de anomalía para cada punto.
     $s(x) = 2^{(-E[h(x)] / c(\text{sample\_size}))}$ 
    Score cercano a 1 → anomalía. Cercano a 0.5 → normal.

    Args:
        X (np.ndarray): Puntos a evaluar, shape (m, n).

    Returns:
        np.ndarray: Scores en (0, 1), shape (m,).
    """
    try:
        scores = []
        c_n = self._c(self.sample_size)
        for x in X:
            avg_len = np.mean(

```

```

        [self._path_length(x, t, 0) for t in self.trees]
    )
    scores.append(2 ** (-avg_len / c_n))
    return np.array(scores)
except Exception as e:
    logger.error("Error en IsolationForest.anomaly_score: %s", e)
    raise

```

```

# --- Prueba de Laboratorio ---

```

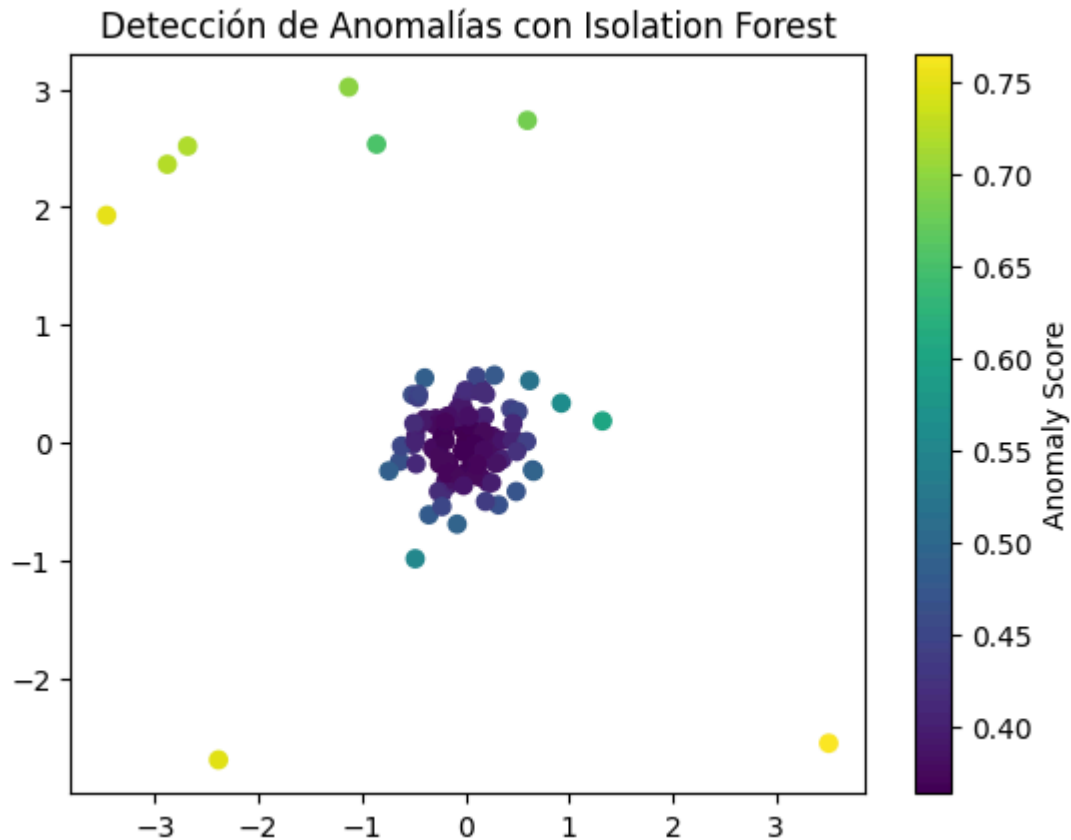
```

if __name__ == "__main__":
    # Generar datos normales
    X_normal = 0.3 * np.random.randn(100, 2)
    # Generar anomalías (puntos alejados)
    X_outliers = np.random.uniform(low=-4, high=4, size=(10, 2))
    X = np.r_[X_normal, X_outliers]

    iforest = IsolationForest(n_trees=100, sample_size=64)
    iforest.fit(X)
    scores = iforest.anomaly_score(X)

    # Visualizar
    import matplotlib.pyplot as plt
    plt.scatter(X[:, 0], X[:, 1], c=scores, cmap='viridis')
    plt.colorbar(label='Anomaly Score')
    plt.title("Detección de Anomalías con Isolation Forest")
    plt.show()

```



Red Básica CNN

Un esqueleto funcional de una CNN muy básica (para MNIST, por ejemplo). Está simplificado el Backpropagation de la convolución para que sea legible, pero el concepto es el mismo.

Nota para la clase: Este código es didáctico. Para producción, nunca lo usaríamos porque NumPy corre en CPU y las CNN necesitan GPUs.

```
import numpy as np

class CapaConvolucion:
    """
    Capa convolucional: aplica num_filtros kernels sobre una imagen 2D.
    Operación: salida[i,j,f] = Σ región[i,j] * filtros[f]

    Args:
        num_filtros (int): Número de filtros aprendibles.
        tam_kernel (int): Tamaño del kernel (tam_kernel x tam_kernel).
    """

    def __init__(self, num_filtros, tamaño_kernel):
        self.num_filtros = num_filtros
        self.tamaño_kernel = tamaño_kernel
```

```

# Inicialización He simplificada
self.filtros = (
    np.random.randn(num_filtros, tamaño_kernel, tamaño_kernel)
    / (tamaño_kernel ** 2)
)

def _regiones(self, imagen):
    """Generador de regiones (tam_kernel x tam_kernel) de la imagen."""
    h, w = imagen.shape
    for i in range(h - self.tamaño_kernel + 1):
        for j in range(w - self.tamaño_kernel + 1):
            yield imagen[i:i+self.tamaño_kernel, j:j+self.tamaño_kernel],
i, j

def forward(self, entrada):
    """
    Paso forward: convolución de entrada con cada filtro.

    Args:
        entrada (np.ndarray): Imagen 2D, shape (H, W).

    Returns:
        np.ndarray: Mapa de características, shape (H', W', F).
    """
    try:
        self.ultima_entrada = entrada
        h, w = entrada.shape
        out_h = h - self.tamaño_kernel + 1
        out_w = w - self.tamaño_kernel + 1
        salida = np.zeros((out_h, out_w, self.num_filtros))
        for region, i, j in self._regiones(entrada):
            salida[i, j] = np.sum(region * self.filtros, axis=(1, 2))
        return salida
    except Exception as e:
        logger.error("Error en CapaConvolucion.forward: %s", e)
        raise

def backward(self, d_L_d_salida, learning_rate):
    """
    Paso backward: actualiza filtros con el gradiente recibido.

    Args:
        d_salida (np.ndarray): Gradiente entrante, shape (H', W', F).
        lr (float): Tasa de aprendizaje.

    Returns:
        None
    """
    try:
        d_filtros = np.zeros(self.filtros.shape)
        for region, i, j in self._regiones(self.ultima_entrada):
            for f in range(self.num_filtros):
                d_filtros[f] += region * d_L_d_salida[i, j, f]

```

```

        self.filtros -= learning_rate * d_filtros
    except Exception as e:
        logger.error("Error en CapaConvolucion.backward: %s", e)
        raise

class CapaMaxPooling:
    """
    Capa Max Pooling: reduce dimensiones tomando el máximo por región.
    Proporciona invarianza a pequeñas traslaciones.

    Args:
        tam_pool (int): Tamaño de la ventana de pooling.
    """

    def __init__(self, tamaño_pool=2):
        self.tamaño_pool = tamaño_pool

    def _regiones(self, imagen):
        """Generador de regiones de pooling."""
        h, w, _ = imagen.shape
        for i in range(h // self.tamaño_pool):
            for j in range(w // self.tamaño_pool):
                region = imagen[
                    i*self.tamaño_pool:(i+1)*self.tamaño_pool,
                    j*self.tamaño_pool:(j+1)*self.tamaño_pool
                ]
                yield region, i, j

    def forward(self, entrada):
        """
        Paso forward: máximo de cada ventana tam_pool x tam_pool.

        Args:
            entrada (np.ndarray): Shape (H, W, F).

        Returns:
            np.ndarray: Shape (H//p, W//p, F).
        """
        try:
            self.ultima_entrada = entrada
            h, w, f = entrada.shape
            salida = np.zeros((h // self.tamaño_pool, w // self.tamaño_pool,
f))

                for region, i, j in self._regiones(entrada):
                    salida[i, j] = np.amax(region, axis=(0, 1))
                return salida
        except Exception as e:
            logger.error("Error en CapaMaxPooling.forward: %s", e)
            raise

    def backward(self, d_salida):
        """

```

Paso backward: solo el píxel máximo recibe el gradiente.

Args:

d_salida (np.ndarray): Gradiente entrante, shape (H', W', F).

Returns:

np.ndarray: Gradiente hacia la capa anterior, misma shape que ultima_entrada.

"""

try:

```
d_entrada = np.zeros(self.ultima_entrada.shape)
for region, i, j in self._regiones(self.ultima_entrada):
    h, w, f = region.shape
    amax = np.amax(region, axis=(0, 1))
    for i2 in range(h):
        for j2 in range(w):
            for f2 in range(f):
                if region[i2, j2, f2] == amax[f2]:
                    d_entrada[
                        i*self.tamaño_pool+i2,
                        j*self.tamaño_pool+j2,
                        f2
                    ] = d_salida[i, j, f2]
```

return d_entrada

except Exception as e:

logger.error("Error en CapaMaxPooling.backward: %s", e)

raise

class CapaSoftmax:

"""

Capa densa + Softmax para clasificación multiclase.

Softmax: $P(\text{clase } k) = e^{z_k} / \sum e^{z_j}$

Args:

tam_entrada (int): Dimensión del vector aplanado.

tam_salida (int): Número de clases.

"""

def __init__(self, tamaño_entrada, tamaño_salida):

self.pesos = np.random.randn(tamaño_entrada, tamaño_salida) /
tamaño_entrada

self.biases = np.zeros(tamaño_salida)

def forward(self, entrada):

"""

Aplana entrada, multiplica por pesos y aplica softmax.

Args:

entrada (np.ndarray): Tensor de cualquier shape.

Returns:

np.ndarray: Probabilidades por clase, shape (tam_salida,).

```

"""
try:
    self.ultima_shape = entrada.shape
    self.ultima_entrada = entrada.flatten()
    totales = np.dot(self.ultima_entrada, self.pesos) + self.biases
    exp = np.exp(totales)
    self.ultima_salida = exp / np.sum(exp)
    return self.ultima_salida
except Exception as e:
    logger.error("Error en CapaSoftmax.forward: %s", e)
    raise

def backward(self, d_L_d_salida, learning_rate):
    """
    Actualiza pesos/biases y retorna gradiente hacia la capa anterior.

    Args:
        d_salida (np.ndarray): Gradiente de la pérdida, shape (C,).
        lr (float): Tasa de aprendizaje.

    Returns:
        np.ndarray: Gradiente hacia la capa anterior (misma shape
                    que ultima_entrada antes de aplanar).
    """
    try:
        for i, grad in enumerate(d_L_d_salida):
            if grad == 0:
                continue
            t_exp = np.exp(self.ultima_salida)
            S = np.sum(t_exp)
            d_out = -t_exp[i] * t_exp / (S ** 2)
            d_out[i] = t_exp[i] * (S - t_exp[i]) / (S ** 2)
            d_t = grad * d_out
            self.pesos -= learning_rate * np.outer(self.ultima_entrada,
d_t)

            self.biases -= learning_rate * d_t
            return np.dot(self.pesos, d_t).reshape(self.ultima_shape)
    except Exception as e:
        logger.error("Error en CapaSoftmax.backward: %s", e)
        raise

# =====
# 4. Probando la CNN "Frankenstein"
# =====
if __name__ == "__main__":
    # Creamos una imagen sintética de 28x28 (tipo MNIST)
    imagen = np.random.rand(28, 28)
    etiqueta = np.zeros(10)
    etiqueta[5] = 1 # Supongamos que es el número '5'

    # Instanciamos nuestras capas
    conv = CapaConvolucion(num_filtros=8, tamaño_kernel=3)
    pool = CapaMaxPooling(tamaño_pool=2)

```

```

# 28-3+1 = 26. 26/2 = 13. 13*13*8 = 1352 neuronas de entrada densa
softmax = CapaSoftmax(tamaño_entrada=13*13*8, tamaño_salida=10)

# --- FORWARD PASS ---
out_conv = conv.forward(imagen)
out_pool = pool.forward(out_conv)
prediccion = softmax.forward(out_pool)
print("Predicción (probabilidades):", np.round(prediccion, 2))

# --- CALCULO DE PÉRDIDA ---
loss = -np.log(prediccion[np.argmax(etiqueta)])
print("Pérdida (Cross-Entropy):", loss)

# --- BACKWARD PASS ---
# Gradiente inicial del error con respecto a la salida
gradient = np.zeros(10)
gradient[np.argmax(etiqueta)] = -1 / prediccion[np.argmax(etiqueta)]

grad_softmax = softmax.backward(gradient, learning_rate=0.01)
grad_pool = pool.backward(grad_softmax)
conv.backward(grad_pool, learning_rate=0.01)
print("Pesos actualizados. ¡La CNN aprendió un poquito!")

Predicción (probabilidades): [0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
Pérdida (Cross-Entropy): 2.290163653122454
Pesos actualizados. ¡La CNN aprendió un poquito!

```

Es el momento perfecto para que observen el valor de la Pérdida (Cross-Entropy). En tu ejecución marcó 2.30; si lo corrieran en un ciclo de entrenamiento real (un loop de muchas épocas), verían cómo ese número baja mientras las probabilidades de la clase correcta suben.

Resumen de las Redes Neuronales Convolucionales (CNN) :

- CNN: El Algoritmo Formal

Una Red Neuronal Convolucional es un modelo de aprendizaje profundo diseñado para procesar datos con una topología de cuadrícula conocida (como imágenes). Su arquitectura formal se basa en tres tipos de capas:

 - Capa de Convolución (
 -):
 - Utiliza un conjunto de filtros (kernels) aprendibles.
 - Propiedad Matemática: Realiza el producto escalar entre los pesos del filtro y una pequeña región de la entrada, permitiendo el compartimiento de pesos (weight sharing).
 - Capa de Activación:
 - Usualmente ReLU (), que introduce no linealidad al sistema.
 - Capa de Pooling (
 -):
 - Reduce la dimensionalidad espacial (ancho x alto) para disminuir el número de parámetros y controlar el sobreajuste.

- El Max Pooling selecciona el valor máximo de una región, proporcionando invarianza a pequeñas traslaciones.
- Capas Totalmente Conectadas (
-):
 - Aplanan el volumen de datos para realizar la clasificación final mediante una función Softmax.
- Métricas de Complejidad
 - Complejidad en Tiempo (Entrenamiento):
 - , donde D es la profundidad, F es el número de filtros, K es el tamaño del kernel y S es el tamaño de la imagen de salida.
 - Complejidad en Espacio:
 - . Es notablemente eficiente ya que los parámetros no dependen del tamaño de la imagen de entrada, sino del tamaño de los filtros.

Comentario para el Cierre de Clase

Lo que acaban de hacer manualmente —calcular el gradiente para actualizar los filtros— es el alma de la visión artificial moderna. Este mismo principio, escalado a millones de parámetros y capas, es lo que permite que una IA reconozca rostros o asista en diagnósticos médicos por imagen.

Q-Learning

```
import numpy as np

class QLearningAgente:
    """
    Q-Learning: aprendizaje por refuerzo tabulado con  $\epsilon$ -greedy.
    Ecuación de Bellman:
        
$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$


    Args:
        n_estados (int): Número de estados del entorno.
        n_acciones (int): Número de acciones disponibles.
        alpha (float): Tasa de aprendizaje.
        gamma (float): Factor de descuento.
        epsilon (float): Probabilidad de exploración.
    """

    def __init__(
        self, n_estados, n_acciones,
        alpha=0.1, gamma=0.9, epsilon=0.1
    ):
        self.q_table = np.zeros((n_estados, n_acciones))
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon

    def elegir_accion(self, estado):
```

```

"""
Selecciona acción con política  $\epsilon$ -greedy.

Args:
    estado (int): Estado actual del agente.

Returns:
    int: Índice de la acción elegida.
"""
try:
    if np.random.uniform(0, 1) < self.epsilon:
        return np.random.choice(self.q_table.shape[1]) # explorar
    return np.argmax(self.q_table[estado])             # explotar
except Exception as e:
    logger.error("Error en elegir_accion: %s", e)
    raise

def aprender(self, estado, accion, recompensa, proximo_estado):
    """
    Actualiza Q(s,a) con la ecuación de Bellman.

    Args:
        estado (int): Estado actual s.
        accion (int): Acción tomada a.
        recompensa (float): Recompensa recibida r.
        proximo_estado (int): Estado siguiente s'.

    Returns:
        None
    """
    try:
        q_actual = self.q_table[estado, accion]
        mejor_q_siguiente = np.max(self.q_table[proximo_estado])
        #  $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$ 
        nuevo_q = q_actual + self.alpha * (
            recompensa + self.gamma * mejor_q_siguiente - q_actual
        )
        self.q_table[estado, accion] = nuevo_q
    except Exception as e:
        logger.error("Error en aprender: %s", e)
        raise

# --- Ejemplo de Uso ---
if __name__ == "__main__":
    # Supongamos un mundo de 5 estados y 2 acciones (Izquierda, Derecha)
    agente = QLearningAgente(n_estados=5, n_acciones=2)

    # Simulación de un paso
    estado_actual = 0
    accion = agente.elegir_accion(estado_actual)
    recompensa = 1 # El agente encontró un premio
    proximo_estado = 1

```

```

    agente.aprender(estado_actual, accion, recompensa, proximo_estado)
    print("Q-Table Actualizada:\n", agente.q_table)

```

Q-Table Actualizada:

```

[[0.1 0. ]
 [0.  0. ]
 [0.  0. ]
 [0.  0. ]
 [0.  0. ]]

```

```

if __name__ == "__main__":
    # Mundo de 5 estados (0 es el inicio, 4 es la meta con premio)
    agente = QLearningAgente(n_estados=5, n_acciones=2)

    # Entrenamos por 1000 episodios
    for episodio in range(5000):
        estado = 0 # Empezar siempre al inicio
        while estado < 4: # Hasta llegar a la meta
            accion = agente.elegir_accion(estado)

            # Definimos la lógica del mundo:
            # Acción 1 (derecha) avanza, Acción 0 (izquierda) retrocede o
            # se queda igual
            proximo_estado = min(4, estado + 1) if accion == 1 else max(0,
            estado - 1)

            # Recompensa: Solo si llega al estado 4
            recompensa = 10 if proximo_estado == 4 else -1 # Penalizamos por
            perder tiempo

            agente.aprender(estado, accion, recompensa, proximo_estado)
            estado = proximo_estado

        print("Q-Table con Conocimiento (Tras 1000 intentos):")
        print(agente.q_table)

```

Q-Table con Conocimiento (Tras 1000 intentos):

```

[[ 3.122  4.58 ]
 [ 3.122  6.2  ]
 [ 4.58   8.   ]
 [ 6.2    10.  ]
 [ 0.     0.   ]]

```

Explicación:

- La Q-Table es su mapa: Al principio está vacía.
- La Recompensa se propaga: Con el tiempo, el valor de estar en el estado 3 (cerca del premio) sube, y eso hace que el valor del estado 2 también suba, y así sucesivamente.
- Convergencia: Al final, la tabla les dirá claramente qué acción tomar en cada cuadro para ganar.

